

Prague University of Economics and Business
Faculty of Informatics and Statistics



Regularization methods in Probabilistic Neural Networks and General Regression Neural Networks

BACHELOR THESIS

Study program: Mathematical Methods in Economics

Specialization: Data Analyses and Modeling

Author: Grigorii Vinogradov

Supervisor: Ing. Karel Šafr, Ph.D.

Prague, June 2025

Declaration

I declare that I wrote this bachelor thesis independently using the sources and literature listed in the bibliography. In addition, generative AI was used to check vocabulary, grammar, and style. It also helped to find additional relevant sources of information, books, and articles that were used and cited when writing this text. Also, GenAI was used to assist with LaTeX and BibTeX syntax and creating the correct BibTeX citations according to APA style.

Acknowledgements

I would like to express my gratitude my supervisor Ing. Karel Šafr, Ph.D., for his patience, valuable advices and honest feedback throughout this thesis. I also appreciate my family for their precious support and motivation.

Abstrakt

Tato bakalářská práce se zabývá aplikací regularizačních technik v general regression neural networks (GRNN) a probabilistic neural networks (PNN). Regularizace L1 a L2 byla integrována do architektur GRNN a PNN pomocí Bayesovské formulace odhadu parametrů. Na rozdíl od standardního přístupu jádrového odhadu hustoty (KDE) byly v této práci modely přeformulovány pomocí Gaussovských směsí pro odhad hustoty, což umožnilo modelům přejít z neparametrické formulace na parametrickou s minimálními změnami v interpretaci. Modely byly implementovány v jazyce Python a jejich výkonnost byla vyhodnocena na reálných datových sadách. V regresních úlohách vedla regularizace L1 a L2 ke zlepšení výkonnosti modelů se snížením střední kvadratické chyby v rozmezí 3,02% až 4,04%. V klasifikačních úlohách se přesnost zlepšila v rozmezí od 0,36% do 4,44%. V některých datových sadách však bylo pozorováno mírné snížení výkonu, což naznačuje, že účinek regularizace závisí na datech.

Klíčová slova

General Regression Neural Networks, Probabilistic Neural Networks, regularizace, strojové učení

Abstract

This thesis investigates the application of regularization techniques in general regression neural networks (GRNN) and probabilistic neural networks (PNN). L1 and L2 regularization has been integrated into GRNN and PNN architectures using a Bayesian formulation of parameter estimation. In contrast to the standard kernel density estimation (KDE) approach, in this work the models were reformulated using a mixture of Gaussians for density estimation, allowing the models to move from a non-parametric to a parametric formulation with minimal changes in interpretation. The models were implemented in Python and their performance was evaluated on real-world datasets. In regression tasks, L1 and L2 regularization led to improvements in model performance, with mean square error reductions in the range of 3,02% to 4,04%. In classification tasks, accuracy improvements ranged from 0,36% to 4,44%. However, a slight performance decrease was observed in some datasets, suggesting that the effect of regularization is data-dependent.

Keywords

General Regression Neural Networks, Probabilistic Neural Networks, regularization methods, machine learning

List of Figures

1.1	Schematic diagram of a PNN	18
1.2	Schematic diagram of a GRNN	19
4.1	Visualization of the training samples and the input vector	36
4.2	Effect of regularization type on training pattern's b_i parameter.	38
4.3	Estimated density distribution with KDE (Gaussian kernel).	39
4.4	Estimated density distribution with mixture of Gaussians (L1 regularized MAP).	39
4.5	Estimated density distribution with mixture of Gaussians (L2 regularized MAP).	40
4.6	Decision boundary for classification (classic PNN).	41
4.7	Decision boundary for classification (L1 regularized MAP).	41
4.8	Decision boundary for classification (L2 regularized MAP).	42
4.9	Comparison of initial, L1 and L2 estimated weights.	43
5.1	Class Separation in 2D PCA Space	46
5.2	Decision boundaries in 2D PCA Space for Parkinson's dataset	47
5.3	Regression visualization in 2D PCA Space	49
5.4	Learning curve for Diabetes dataset.	50
5.5	Learning curve for Concrete Compressive Strength dataset.	51
5.6	The effect of regularization strength on MSE	52

List of Tables

4.1	Demonstration synthetic dataset.	36
4.2	Summary of the L1 and L2 regularization outcomes.	37
5.1	Models evaluation on selected datasets for classification problem	45
5.2	Models evaluation on selected datasets for regression problem	48

Contents

Introduction	10
1 Theoretical basis of PNN and GRNN	11
1.1 Bayes optimal decision strategy	11
1.2 General Regression	13
1.3 Distance metrics and kernel density estimation	15
1.3.1 Distance mterics	15
1.3.2 Kernel functions	16
1.4 PNN and GRNN architecture	17
2 Fundamental Methods of Regularization	21
2.1 L1 regularization	21
2.2 L2 regularization	22
2.3 Dropout	22
2.4 Bayesian Perspective on L1 and L2 regularization	23
2.4.1 Maximum likelihood estimation	23
2.4.2 Maximum a posteriori estimation	24
3 Regularization in PNN and GRNN	27
3.1 Weights interpretation in PNN and GRNN	27
3.2 Mixture of Gaussians	28
3.2.1 Softmax transformation of a parameters	29
3.2.2 Prior distribution definition	29
3.2.3 General form of MAP estimation	29
3.2.4 MAP estimation in PNN	30
3.2.5 MAP estimation in GRNN	31
3.2.6 Hyperparameter τ	32
3.3 Analytical intractability and optimization strategy	33
3.3.1 Gradient Descend	33
3.3.2 Adam	34
4 Demonstration of regularization in PNN and GRNN on synthetic data	35
4.1 Data	35
4.2 Models	36
4.3 Results of parameter estimation	37
4.3.1 Estimated density shape	38
4.3.2 Decision boundaries	40
4.3.3 Weights values comparison	43
5 Evaluation on real datasets	44
5.1 Preprocessing, metrics and model selection	44

5.2	Classification	44
5.3	Regression	47
5.4	Learning curve	50
5.5	The effect of regularization strength on model error	51
	Conclusion	53
	Bibliography	55
A	Algorithms of optimization	58

Introduction

Machine learning is one of the most rapidly developing disciplines in the fields of mathematics and statistics. In recent years, neural networks have demonstrated great results in the accuracy of predictions. The best known and most used type of neural networks are the Back Propagation Networks (BPN). BPN may not be efficient in some cases when the aim is to explore new data and the task requires frequent retraining and evaluation with small datasets. Furthermore, the BPN approach to estimating class statistics can be considered heuristic, relying more on optimization techniques than on statistical principles. (Specht, 1990)

Probabilistic Neural Networks (PNN) and General Regression Neural Networks (GRNN) are feedforward neural networks, introduced by Specht (1990, 1991). Those models are fundamentally based on probability and statistical principles. Among the main advantages of using PNN and GRNN over BPN in some cases is their non-parametric design. Unlike BPN, which requires estimation of a large number of parameters and may depend on initial conditions, PNN and GRNN form a solution based on the statistical characteristics of the training sample without the need for iterative weight adjustment. In addition, such models have a high generalization ability that is often as good as and sometimes superior to BPN, especially with limited training data. As will be demonstrated in this work, although there has been research on regularization in PNN and GRNN, the proposed and most commonly used methods are typically designed specifically for these types of network.

This bachelor thesis aims to study and practical implementation of regularization techniques in GRNN and PNN. These models, known for their ability to approximate complex non-linear dependencies, present unique opportunities and challenges when it comes to controlling overfitting. The main goal of this paper is to study existing regularization methods, adapt and apply generally known methods of regularization to GRNN and PNN architectures, implement these methods in a programming language, and evaluate their performance on real datasets.

The motivation for this thesis was not only a strong interest in machine learning and probability theory but also the opportunity to investigate methods with significant practical potential. GRNN and PNN are known for their strong generalization abilities. However, their performance can be potentially improved by applying regularization techniques. This thesis begins with a comprehensive review of the theoretical and mathematical foundations of GRNNs, PNNs and regularization methods. Subsequently, selected regularization methods are implemented from the ground up and integrated into the GRNN and PNN models. The effectiveness of these models is then evaluated through empirical evaluation on real world datasets.

1. Theoretical basis of PNN and GRNN

1.1 Bayes optimal decision strategy

As outlined by Specht (1990), in contrast to BPN, which rely on empirical optimization of a loss function through iterative gradient descent, PNN implement the Bayes optimal decision strategy, leveraging probabilistic inference for classification. In this chapter, a formal derivation of the Bayes optimal decision strategy and its relevance to PNN classification model is presented.

To establish the probabilistic classification framework used in probabilistic neural networks, we begin by formally defining the necessary elements of Bayesian decision theory. Let X denote the observation. Associated with these inputs are known class labels $\{y_1, y_2, \dots, y_n\} \subset Y$, where $Y = \{\theta_A, \theta_B\}$ denotes the space of possible class labels (target values). (Specht, 1990)

The functions $f_A(X)$ and $f_B(X)$ denote the class-conditional probability density functions (PDFs) for classes θ_A and θ_B , respectively. These represent the likelihoods $P(X|\theta_A)$ and $P(X|\theta_B)$ in the context of Bayes' theorem. The prior probabilities of the classes are denoted by $h_A = P(\theta_A)$ and $h_B = P(\theta_B)$, with the constraint $h_B = 1 - h_A$. (Specht, 1990)

As specified by Specht (1990), in addition to likelihood and prior, the Bayes decision strategy also requires a loss function $L(d(X), \theta)$, which defines a penalty for cases where the decision function $d(X)$ assigns the input X to a class different from its true label θ . The loss function is defined as follows:

$$L(d(X), \theta) = \begin{cases} l_A, & \text{if } d(X) = \theta_B \text{ and } \theta = \theta_A \\ l_B, & \text{if } d(X) = \theta_A \text{ and } \theta = \theta_B \\ 0, & \text{if } d(X) = \theta \end{cases}$$

This formulation forms the underlying basis for the optimal Bayesian decision rule used in PNN, which selects the class that minimizes the expected risk based on likelihood, priors, and loss values.

The following discussion in this section is based on Chapter 2 (Bayesian decision theory) of (Duda et al., 2000). According to Duda et al. (2000), the object is to derive a decision function d that optimally maps each x_i to its corresponding y_i by minimizing the expected risk $R(d|X)$ of a decision rule. The expected risk is a measure that quantifies the average cost or penalty we expect to incur when making decisions based on a given rule, taking into account the uncertainty of the outcomes. In the context of classification, it represents

the weighted average of the losses associated with choosing a particular class for an object, where the weights are the probabilities of each possible true class given the observed data. Essentially, it tells us how ‘wrong’ we expect our decision to be on average across all possible scenarios, balancing the consequences of errors with how likely those errors are to happen. Expected risk for observation X in classification with two classes θ_A and θ_B (Duda et al., 2000):

$$R(d|X) = L(d(X), \theta_A)P(\theta_A|X) + L(d(X), \theta_B)P(\theta_B|X)$$

The overall risk, associated with the decision function d is (Duda et al., 2000):

$$R(d) = \int_X R(d|X)P(X)dx$$

As outlined by Duda et al. (2000), the optimal decision rule $d : X \rightarrow Y$ to be derived is the one that, on average, yields the minimum penalty. Formally, it is defined as the function that minimizes the expected risk $R(d)$.

$$\begin{aligned} d &= \arg \min_d R(d) \\ &= \arg \min_d \int_X R(d|X)P(X)dx \end{aligned}$$

From the formulation of the overall risk $R(d)$ it is clear that minima will be achieved if, for each X , $d(X)$ is assigned so that it minimizes $R(d|X)$ (Duda et al., 2000).

$$d(X) = \arg \min_{y \in Y} R(y|X)$$

We compute the risk, associated with all possible decisions d and observation X . And then select the decision $d(X)$, for which $R(d|X)$ is minimum. (Duda et al., 2000)

Following the work of Duda et al. (2000), for clarity, we will assume that the classification problem consists of two classes and derive a finite decision rule for this particular case. If $d(X) = \theta_A$:

$$R(\theta_A|X) = L(\theta_A, \theta_A)P(\theta_A|X) + L(\theta_A, \theta_B)P(\theta_B|X)$$

If $d(X) = \theta_B$:

$$R(\theta_B|X) = L(\theta_B, \theta_A)P(\theta_A|X) + L(\theta_B, \theta_B)P(\theta_B|X)$$

According to Specht (1990), loss associated with a correct classification is zero, which is mostly assumed in PNN, the corresponding terms are eliminated: - If $d(X) = \theta_A$:

$$R(\theta_A|X) = L(\theta_A, \theta_B)P(\theta_B|X)$$

- If $d(X) = \theta_B$:

$$R(\theta_B|X) = L(\theta_B, \theta_A)P(\theta_A|X)$$

The posterior probability $P(\theta_i|X)$ is then expressed in terms of the prior probability and likelihood using Bayes’ theorem, with these quantities to be approximated using PNN in subsequent sections:

$$\begin{aligned} p(\theta|x) &= \frac{p(x|\theta)p(\theta)}{p(x)} \\ P(\theta_i|X) &= \frac{f_{\theta_i}(X)h_i}{P(X)} \end{aligned}$$

The decision rule can thus be reformulated as:

$$\begin{aligned} d(X) &= \begin{cases} \theta_A : L(\theta_B, \theta_A) \frac{f_A(X)h_A}{P(X)} > L(\theta_A, \theta_B) \frac{f_B(X)h_B}{P(X)} \\ \theta_B : L(\theta_B, \theta_A) \frac{f_A(X)h_A}{P(X)} < L(\theta_A, \theta_B) \frac{f_B(X)h_B}{P(X)} \end{cases} \\ &= \begin{cases} \theta_A : L(\theta_B, \theta_A) f_A(X)h_A > L(\theta_A, \theta_B) f_B(X)h_B \\ \theta_B : L(\theta_B, \theta_A) f_A(X)h_A < L(\theta_A, \theta_B) f_B(X)h_B \end{cases} \end{aligned}$$

The final form of the Bayes decision rule will be (Duda et al., 2000; Specht, 1990):

$$d(X) = \begin{cases} \theta_A : L(\theta_B, \theta_A) f_A(X)h_A > L(\theta_A, \theta_B) f_B(X)h_B \\ \theta_B : L(\theta_B, \theta_A) f_A(X)h_A < L(\theta_A, \theta_B) f_B(X)h_B \end{cases}$$

According to Specht (1990), for the multi-class classification, where: $Y = \{\theta_1, \dots, \theta_n\}$, the decision function is defined as:

$$d(X) = \theta_i \text{ if } \forall j, j \neq i : h_{\theta_i} l_{\theta_i} f_{\theta_i}(X) > h_{\theta_j} l_{\theta_j} f_{\theta_j}(X)$$

Subsequently, the application of PNN requires the estimation of the class-conditional probabilities $P(X|\theta_A)$ and $P(X|\theta_B)$. This will be covered in Sec. 1.3

1.2 General Regression

The GRNN model, introduced by Specht (1991), also relies on the statistical principles of estimating the target values. It uses the general regression principles of estimating the conditional mean of the target variable y denoted as $\mathbb{E}[y|X]$. Let x denote the input, and let $\{x_1, x_2, \dots, x_n\} \subset X$ be a finite training set sampled from this space. Correspondingly, let $y = \mathbb{R}$ represent the output (target) space for regression tasks and let $\{y_1, y_2, \dots, y_n\} \subset Y$ denote the associated set of observed target values (labels) for training instances.

Let $f(x, y)$ denote the joint probability density function of the random variables x and y . Then, the conditional expectation of the target variable given an input $x \in X$ is defined as (Specht, 1991):

$$\mathbb{E}[y | x] = \frac{\int_{-\infty}^{\infty} y f(x, y) dy}{\int_{-\infty}^{\infty} f(x, y) dy}, \quad (1.1)$$

where x represents the particular observed value of the input object space X .

The primary goal of GRNN is to predict the continuous target variable y for a given input x by estimating its conditional mean $\mathbb{E}[y | x]$. Unlike traditional regression methods that fit parametric models (e.g., linear regression) or BPN, which uses optimization over a loss function, GRNN adopts a non-parametric approach, relying on the underlying data distribution

represented by $f(x, y)$. This makes GRNN particularly effective in capturing complex, non-linear relationships without assuming a specific functional form. In practice, the true joint density $f(x, y)$ is unknown and must be estimated from the training set. GRNN achieves this through a kernel-based approximation, treating each training sample as a contribution to the density estimate. The density estimation will be reviewed in Sec. 1.3. According to Specht (1991), the joint probability is estimated as:

$$\hat{f}(x, y) = \frac{1}{(2\pi)^{(p+1)/2} \sigma^{p+1}} \cdot \frac{1}{n} \sum_{i=1}^n \exp\left(-\frac{(\|x - x_i\|_2^2)}{2\sigma^2}\right) \cdot \exp\left(-\frac{(y - y_i)^2}{2\sigma^2}\right) \quad (1.2)$$

As described by Specht (1991), the probability estimate $\hat{f}(x, y)$ can be interpreted as follows: it models each point from samples x_i and y_i as a source of a local probability distribution of width σ , and then sums the contributions of all such sources to obtain an overall approximation of the probability distribution. Substituting the joint probability with the estimate into Eq. (1.1) yields the final form of the conditional mean estimation, which is the estimation for the target variable $\hat{y}$:

$$\hat{y}(x) = \frac{\sum_{i=1}^n \exp\left(-\frac{(\|x - x_i\|_2^2)}{2\sigma^2}\right) \int_{-\infty}^{\infty} y \exp\left(-\frac{(y - y_i)^2}{2\sigma^2}\right) dy}{\sum_{i=1}^n \exp\left(-\frac{(\|x - x_i\|_2^2)}{2\sigma^2}\right) \int_{-\infty}^{\infty} \exp\left(-\frac{(y - y_i)^2}{2\sigma^2}\right) dy} \quad (1.3)$$

$$= \frac{\sum_{i=1}^n y_i \cdot \exp\left(-\frac{(\|x - x_i\|_2^2)}{2\sigma^2}\right)}{\sum_{i=1}^n \exp\left(-\frac{(\|x - x_i\|_2^2)}{2\sigma^2}\right)}, \quad (1.4)$$

where $\|x - x_i\|_2^2$ is a square of the euclidean distance (see Sec. (1.3.1)) between the vectors x and x_i , which is represented with the vector operation as $(x - x_i)^\top (x - x_i)$ in (Specht, 1991). The exponents in Eq. (1.4) are effectively the unnormalized Gaussian kernels defined in Sec. 1.3 Eq. (1.7). Generally, any kernel function can be used in conditional mean estimation (Specht, 1991):

$$\hat{y}(x) = \frac{\sum_{i=1}^n y_i K(x, x_i)}{\sum_{i=1}^n K(x, x_i)}, \quad (1.5)$$

where $K(x, x_i)$ is a kernel function that measures the similarity between the input x and the training point x_i . The denominator $\sum_{i=1}^n K(x, x_i)$ approximates the marginal density $f(x)$ or $\int_{-\infty}^{\infty} f(x, y) dy$, while the numerator estimates the weighted contribution of the target values y_i to the joint density $f(x, y)$. (Specht, 1991)

Further in this work, the x_i notation for points or vectors from the training set will be replaced by W_i as a pattern to ensure consistency of notation between PNN and GRNN.

Since the kernel density estimation method employs distance metrics calculation, this estimate can be interpreted as the weighted mean of the observed target values y_i , where the weights are effectively the distance from the input parameter to other observed patterns. (Specht, 1991)

1.3 Distance metrics and kernel density estimation

Kernel density estimation (KDE) is a statistical non-parametric technique that employs kernel smoothing to estimate the probability density function (PDF) of a random variable. It was originally introduced independently by Rosenblatt (1956) and Parzen (1962). This method is particularly valuable when the underlying PDF of the random variable is unknown. Probabilistic neural networks and general regression neural networks utilize this technique internally to estimate the PDF, allowing them to make predictions. Let $\{W_1, W_2, \dots, W_n \in \mathbb{R}^p\} \sim IID$ be samples that drawn from an unknown distribution with an unknown PDF f . The objective is to derive an approximation of this PDF, denoted as $\hat{f}$. As described by Chen (2017), formally, the KDE can be defined as:

$$\hat{f}(x) = \frac{1}{n\sigma^p} \sum_{i=1}^n K\left(\frac{x - W_i}{\sigma}\right)$$

where x is the input vector, W_i are training samples and K is the kernel function. According to Phillips and Venkatasubramanian (2011), the kernel function is a non-negative real-valued integrable function, that at any given point x $K(\frac{x-x}{\sigma})$ reaches maxima and it decreases as the distance between W_i and x increases. In the following parts of this work, the notation for the kernel function will be redefined as $K(W_i, x)$ for simplicity.

The kernel function is scaled by the smoothing (or bandwidth) parameter σ . This is a free non-negative parameter that influences the estimation and controls the smoothness of the estimate. The choice of the kernel function plays a critical role in the effectiveness of the estimation. Generally, this parameter should be as small as the data permits, however, the bias-variance tradeoff of the estimate must be carefully considered. In the case when the σ is too large, the estimation will be too biased, which means that the kernel function becomes overly smooth and fails to capture the fine details of the underlying data distribution. Consequently, the model struggles to differentiate between distinct patterns or classes, leading to poor predictive performance and a lack of sensitivity to local variations in the data. On the other extreme, if the σ is too small, the variance of estimation will be huge, consequently, the performance of the result model will be small. (Specht, 1990, 1991)

1.3.1 Distance mterics

Kernel functions typically rely on a distance metric to compute their values based on the distance of data points. The choice of distance metric, kernel function, and bandwidth parameter σ plays a crucial role in determining the bias-variance trade-off in the estimation of kernel density. The distance metric itself is a function that measures the distance between two vectors.

Euclidean distance

Euclidean distance, also known as L2 distance, is one of the most widely used metrics. According to Hastie et al. (2009, p. 509) and Han et al. (2011, p. 72) It is the Euclidean norm of the difference between vector x and vector y in the linear space and have following form:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} = \|x - y\|_2$$

According to Bishop (2006, pp. 427–428), Euclidean distance is sensitive to outliers and scale, meaning that features with larger scales dominate the distance calculation. Therefore, the data should be normalized or standardized before applying the metric.

Manhattan distance

Manhattan distance, or L1 distance, represents the sum of absolute differences for each dimension of two vectors, as stated by Han et al. (2011, p. 72):

$$d(x, y) = \sum_{i=1}^n |x_i - y_i| = \|x - y\|_1$$

Unlike Euclidean distance, Manhattan distance does not square differences, making it less sensitive and more robust to outliers.

Thus, by applying different distance metric in the kernel function one can significantly influence the performance of PNN and GRNN.

1.3.2 Kernel functions

Gaussian kernel

In probabilistic neural networks (PNNs), each training example acts as the center of a localized influence region, defined by a kernel function. According to Phillips and Venkatasubramanian (2011) this function assigns a high value near the training point and gradually decays with distance. As mentioned in (Specht, 1995), usually, in the context of PNN and GRNN the Gaussian kernel is used:

$$K(W, x) = \frac{1}{(2\pi\sigma^2)^{p/2}} \exp\left(-\frac{\|W - x\|_2^2}{2\sigma^2}\right), \quad (1.6)$$

where, in the context of PNN and GRNN, x is the input vector, W is a training pattern and p is the dimension of the vectors.

This form is normalized, ensuring that the kernel integrates to 1 and thus can be directly interpreted as a probability density.

In many practical implementations (e.g., classical GRNN/PNN), a simplified unnormalized version is used (Fadel et al., 2016; Specht, 1990):

$$K(W, x) = \exp \left(-\frac{\|W - x\|_2^2}{2\sigma^2} \right) \quad (1.7)$$

This form is sufficient for models in which kernels are utilized in a ratio, where the normalizing constants cancel out. In addition, in (Specht, 1992) mentioned that the same kernel but with the manhattan distance works equally well as the Gaussian kernel:

$$K(W, x) = \exp \left(-\frac{\|W - x\|_1^2}{2\sigma^2} \right)$$

Laplacian kernel

The Laplacian kernel is also often used (Fadel et al., 2016):

$$K(W, x) = \frac{1}{2\sigma} \exp \left(-\frac{\|W - x\|_1}{\sigma} \right), \quad (1.8)$$

and its unnormalized version:

$$K(W, x) = \exp \left(-\frac{\|W - x\|_1}{\sigma} \right) \quad (1.9)$$

As described by Fadel et al. (2016), this kernel is less sensitive to the bandwidth parameter σ .

1.4 PNN and GRNN architecture

The probabilistic neural network is a feedforward neural network. PNN was first introduced by Specht (1990) as a neural network implementation of kernel density estimation for classification tasks, implementing the Bayes optimal decision strategy in a neural network. It is particularly well-suited for problems where probabilistic interpretations of predictions are desired, as it provides not only a class label but also an estimate of the class-conditional probability densities. The standard implementation of a PNN consists of four layers: the input layer, the pattern layer, the summation layer, and the output layer.

The input layer receives the input feature vector x and distributes its components to each node in the Pattern Layer for further processing. (Specht, 1990)

The pattern layer stores the training set $\{W_i, y_i\}$, where each node corresponds to an object from the training set. In PNN, each training sample, feature vector W_i with a corresponding class y_i is considered as a pattern of its class. For a given input feature vector x , the value of a kernel function $K(W_i, x)$ is calculated for each pattern in the pattern layer. The resulting kernel values are then passed to the Summation Layer. (Specht, 1990)

The summation layer contains nodes that represent the estimated probability density functions for each class $\theta_1 \dots \theta_k$. Each node corresponds to a class θ_k and computes the normalized sum of kernel values for all training samples belonging to that class:

$$f_{\theta_k}(x) = \frac{1}{n_k} \sum_{i=1}^{n_k} K(W_i^{\theta_k}, x) \quad (1.10)$$

where $W_i^{\theta_k}$ represents the i^{th} pattern from class θ_k and n_k is the number of observations from class θ_k in the training set. (Specht, 1990)

The output layer makes the final classification decision by selecting the class with the highest estimated probability. It applies the Bayes decision rule to the output of the Summation Layer, choosing the class that maximizes the following expression:

$$\arg \max_{\theta} (h_{\theta} l_{\theta} f_{\theta}(x))$$

The following schematic diagram illustrates the architecture of the PNN in the two-class classification task (Specht, 1990). Highlighting the flow of data through its layers and the role of KDE in estimating PDF.

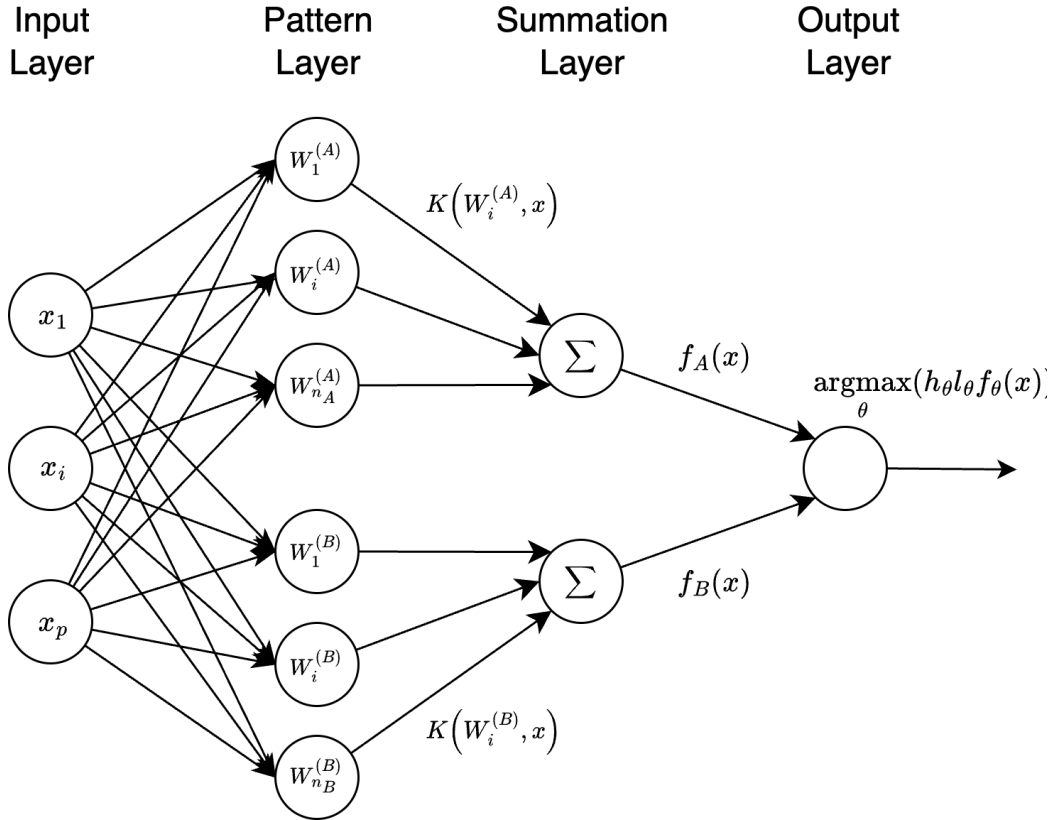


Figure 1.1: Schematic diagram of a PNN

Source and processing: author, inspired by (Specht, 1990)

The general regression neural network, similar to PNN, is a feedforward neural network, also proposed by Specht (1991). This network consists of the same four layers: the input layer,

the pattern layer, the summation layer, and the output layer. The difference from PNN lies in the principle of operation of the last two layers, the summation layer and the output layer. As mentioned above, the general regression neural network relies on the principles of general regression, which consist in deriving the conditional mean of the target variable from seen patterns from the training set. This involves estimating the probability density function of the input vector x . Similarly to PNN, this is done using KDE, for each pattern from the pattern layer the kernel function value $K(W_i, x)$ is computed, which is later used in the summation layer.

In contrast to the Summation Layer in PNN, where the kernel function values are summed for each class independently, in GRNN the nodes in the summation layer represent the numerator and the denominator of the formula for the estimation, which was defined in Eq. (1.5):

$$\hat{y}(x) = \frac{\sum_{i=1}^n y_i K(W_i, x)}{\sum_{i=1}^n K(W_i, x)}$$

The output layer simply divides two outputs from the Summation Layer, providing the final prediction - $\hat{y}(x)$.

The diagram below depicts the GRNN architecture, illustrating the difference between PNN and GRNN and highlighting the role of KDE in estimating probability density functions.

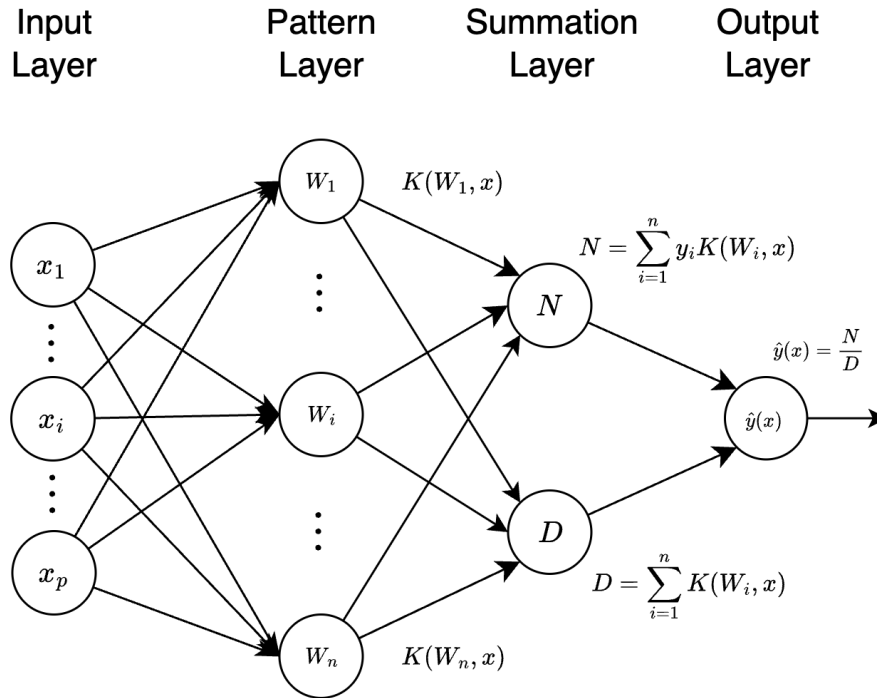


Figure 1.2: Schematic diagram of a GRNN

Source and processing: author, inspired by (Specht, 1991)

To better understand the unique characteristics of PNN and GRNN, it is helpful to compare them with traditional neural networks, such as back propagation networks. According

to Goodfellow et al. (2016), in a typical BPN, the architecture consists of an input layer, one or more hidden layers, and an output layer, where each layer is fully connected to the next. The weights w_{ij} represent the strength of the connection between neurons across layers. These weights are learned during training through backpropagation, where the error between predicted and actual outputs is minimized by iteratively adjusting the weights using gradient descent. The learning process involves optimizing a loss function and often includes regularization techniques to avoid overfitting the model. In contrast to BPN, PNN and GRNN do not have weights in a traditional sense of trainable parameters of the model. Instead, weights information is stored in the training samples or patterns, which are represented by the Pattern Layer. Thus, all the necessary information for the inference is stored in the Pattern Layer, other layers of the network are needed to perform necessary calculations to make the inference. Thus, there is no iterative weight adjustment in PNN and GRNN, those networks require a single pass for training. The training phase consists of storing the training data as patterns in the pattern layer and computing the prior probability. However, computational complexity is postponed to the prediction phase, when the values of the kernel function are calculated and summed to obtain density estimates, which are used to make a final prediction. (Specht, 1990, 1991)

2. Fundamental Methods of Regularization

The concepts of overfitting and generalization of the model are addressed in (Goodfellow et al., 2016, sec. 5.2) and (Murphy, 2022, pp. 120–129). Formally, regularization can be defined as the process of simplification of a statistical or machine learning model by adding complexity penalty to the estimator or performing some heuristic algorithms to make the model more generalized. More specifically, in the case of machine learning, regularization is usually defined as preventing overfitting of the model.

As pointed out in (Goodfellow et al., 2016, sec. 5.2), the overfitted model itself is a model that does not generalize on the training data to predict new, unseen data, but fully relies on the training data, having a high variance in its parameters. Thus, the overfitted model is a model that performs well on training data but at the same time performs worse on new data. Strategies that prevent the model from going into an overfitted state are called regularization.

Regularization has been well studied for various machine learning models, including linear and logistic regression, decision trees, and support vector machines, among others. Numerous regularization techniques for traditional neural networks trained with backpropagation are also well known, such as dropout, L_1 , L_2 , etc. (Goodfellow et al., 2016). However, in the context of probabilistic neural networks and general regression neural networks, regularization techniques have received less attention, despite the potential of these models to achieve strong predictive performance when combined with effective regularization strategies. To understand and develop regularization methods in PNN and GRNN it is necessary to understand regularization in classical neural networks as well.

2.1 L1 regularization

As described in (Goodfellow et al., 2016, sec. 7.1), L1 regularization, also known as Lasso (Least Absolute Shrinkage and Selection Operator) or L1 weight decay, is a method that is based on adding to the loss function the sum of the absolute values of the weights, scaled by a hyperparameter λ , which controls the strength of regularization:

$$\tilde{\mathcal{L}}(w) = \mathcal{L}(w) + \lambda \sum_{i=1}^h |w_i|$$

where $\mathcal{L}(w)$ is a loss function without regularization associated with weights w , which is optimized during maximum likelihood estimation, $\lambda \sum_{i=1}^h |w_i|$ is a regularization term, and $\tilde{\mathcal{L}}(w)$ is the regularized loss function. In (Murphy, 2022, p. 385) the example for the residual sum of squares (or sum of squared errors) loss function, then L1 regularized residual sum of

squares is presented for the linear regression form. We will simplify it for general cases:

$$RSS_{L1}(w) = \sum_{i=1}^n (y_i - f_w(x_i))^2 + \lambda \sum_{i=1}^h |w_i|$$

where y_i is a true value of the dependent variable, f_w is a model with weights w , $f_w(x_i)$ is its prediction of the dependent variable on input x_i .

According to Goodfellow et al. (2016, sec. 7.1) and Murphy (2022, pp. 386–387), applying this method may result in sparse weights w . This regularization may reduce some weights to zero if their effect on the error is small. Thus, this method enables feature selection by effectively excluding less important features from the model.

2.2 L2 regularization

L2 regularization, also known as Ridge or L2 weight decay, is a method that, like L1 regularization, penalizes the magnitude of weights in a model to prevent overfitting. According to Goodfellow et al. (2016, sec. 7.1) In the case of L2 regularization, the penalty term is the sum of the squares of the weights :

$$\tilde{\mathcal{L}}(w) = \mathcal{L}(w) + \lambda \sum_{i=1}^h w_i^2$$

In (Murphy, 2022, p. 381) L2-regularized residual sum of squares loss is considered as an example for linear regression model, we will also generalize it for any model f_w :

$$RSS_{L2}(w) = \sum_{i=1}^n (y_i - f_w(x_i))^2 + \lambda \sum_{i=1}^h w_i^2$$

As it described in (Murphy, 2022, pp. 386–387), unlike L1 regularization, which may drive some weights to exactly zero due to the linear growth of its penalty term, L2 regularization typically does not result in weights being exactly zero. This is because the penalty term in L2 regularization grows quadratically with the weights, corresponding to a spherical constraint in the weight space that smoothly shrinks weights toward zero without forcing them to be exactly zero. This property makes L2 regularization particularly effective in scenarios where all features are expected to contribute to the prediction, as it distributes the penalty more evenly across the weights, leading to smoother and more stable models.

2.3 Dropout

The concept of dropout is also covered in (Goodfellow et al., 2016, sec. 7.12). Dropout is a regularization technique that involves temporarily disabling a subset of nodes in an artificial neural network during each training step. By doing so, the model reduces the risk of overfitting and reduces the level of complexity, thereby achieving the goal of regularization.

Dropout prevents co-adaptation of neurons by forcing the network to learn more robust features that are not overly reliant on specific nodes. The dropout is performed by removing the units from the network by applying the mask on them:

$$\hat{w}_j = \begin{cases} 0, & \text{with } p \\ w_j, & \text{otherwise} \end{cases}$$

During inference, Dropout is not applied and weights are scaled by $1 - p$ to maintain the expected output of each neuron. (Goodfellow et al., 2016)

2.4 Bayesian Perspective on L1 and L2 regularization

To fully understand where the L1 and L2 regularization in the models comes from, Bayesian linear regression models can be considered. In addition, this knowledge will be key to adapting L1 and L2 regularization principles in PNN and GRNN, as these are primarily probability-based models. In this section, the linear regression model will be considered to demonstrate how to derive regularization penalties using probabilistic and statistical principles.

2.4.1 Maximum likelihood estimation

Maximum likelihood estimation (MLE) is the method widely used in estimating parameters. In this section, an example of MLE for a linear regression model is provided. Linear regression from the frequentist perspective is defined as a linear model whose parameters maximize likelihood. According to Murphy (2022, p. 371), the linear regression model in probabilistic terms is expressed as:

$$p(y|x, \beta, \sigma^2) = \mathcal{N}(y|\beta^\top x, \sigma^2)$$

where y is the target variable, $x = [1, x^{(1)}, x^{(2)}, \dots, x^{(p)}]$ is the vector of regressors (feature vector), $\beta = \{\beta_i\}_0^p$ and σ^2 are parameters of the model. The parameter β_0 is the bias or intercept term, because of that we define the vector x with 1 in the first index. (Murphy, 2022, p. 371)

According to Murphy (2022, pp. 114–115), the estimated parameters $\hat{\beta}$ in linear regression are obtained via MLE, or, in other words, by minimizing the negative log likelihood on the training data with the assumption that the training samples are independent. The maximum likelihood estimation for β is given by the formulation provided in (Murphy, 2022, pp. 107–108), where y_i is a known target variable for i^{th} observation and x_i is a feature vector for i^{th}

observation from a dataset of n observations:

$$\hat{\beta}_{MLE} = \arg \max_{\beta} \prod_{i=1}^n p(y_i | x_i, \beta, \sigma^2) \quad (2.1)$$

$$= \arg \max_{\beta} \sum_{i=1}^n \log p(y_i | x_i, \beta, \sigma^2) \quad (2.2)$$

$$= \arg \min_{\beta} - \sum_{i=1}^n \log p(y_i | x_i, \beta, \sigma^2) \quad (2.3)$$

In (Murphy, 2022, pp. 372–373), the negative log-likelihood is presented as:

$$- \sum_{i=1}^n \log p(y_i | x_i, \beta, \sigma^2) = - \sum_{i=1}^n \log \mathcal{N}(y | \beta^\top x, \sigma^2) \quad (2.4)$$

$$= - \sum_{i=1}^n \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(- \frac{(y_i - \beta^\top x_i)^2}{2\sigma^2} \right) \right] \quad (2.5)$$

$$= \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta^\top x_i)^2 + \frac{n}{2} \log(2\pi\sigma^2) \quad (2.6)$$

By substituting the negative log likelihood in Eq. (2.6) into Eq. (2.3) we obtain the following form, as shown in (Murphy, 2022, pp. 372–373):

$$\hat{\beta}_{MLE} = \arg \min_{\beta} \left[\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta^\top x_i)^2 + \frac{n}{2} \log(2\pi\sigma^2) \right] \quad (2.7)$$

$$= \arg \min_{\beta} \sum_{i=1}^n (y_i - \beta^\top x_i)^2 \quad (2.8)$$

The terms $\frac{1}{2\sigma^2}$ and $\frac{n}{2} \log(2\pi\sigma^2)$ are irrelevant constants, which can be ignored while minimizing with respect to β . After dropping the constants, we obtain the residual sum of squares (RSS), which is to be minimized (Murphy, 2022, pp. 372–373):

$$RSS(\beta) = \sum_{i=1}^n (y_i - \beta^\top x_i)^2$$

2.4.2 Maximum a posteriori estimation

As explained in (Murphy, 2022, p. 154), the Bayesian approach to estimating the parameters of a linear regression model differs from the frequentist approach described earlier. In Bayesian approach, we treat each parameter as a random variable, which consequently has its own distribution. In frequentist approach, we do not treat parameters as random variables, but assuming that those are unknown constants.

In addition, it is mentioned in (Murphy, 2022, pp. 120–121) that when trying to estimate parameters using MLE, overfitting often occurs. The reason is that MLE minimizes the loss (RSS in the case of linear regression) on the training dataset, but it does not guarantee that the model will have a good loss on unseen data.

The L1 and L2 regularization employs adding a penalty term to the loss. As presented by Murphy (2022, p. 121), this can be derived if it appears when we consider maximum a

posteriori (MAP) estimation for the parameter estimation. Again, in this section, we will consider the linear regression model as an example.

In (Murphy, 2022, p. 121) the MAP estimation for β parameters is presented:

$$\hat{\beta}_{MAP} = \arg \max_{\beta} p(\beta|\mathcal{D}), \quad (2.9)$$

where $p(\beta|\mathcal{D})$ is the posterior density of the parameters given the observed data $\mathcal{D}$, y and x in context of linear regression.

To derive $\hat{\beta}_{MAP}$, we apply Bayes' theorem and apply the log function:

$$p(\beta|\mathcal{D}) = \frac{p(\mathcal{D}|\beta)p(\beta)}{p(\mathcal{D})},$$

where $p(\mathcal{D}|\beta)$ is the likelihood of the observed responses, $p(\beta)$ is the prior distribution over the parameters, $p(\mathcal{D})$ is the marginal likelihood, which is a normalizing constant and does not depend on parameters. For estimation, this term can be ignored, see (Murphy, 2022, p 121):

$$\hat{\beta}_{MAP} = \arg \max_{\beta} p(\mathcal{D}|\beta)p(\beta) \quad (2.10)$$

$$= \arg \max_{\beta} [\log p(\mathcal{D}|\beta) + \log p(\beta)] \quad (2.11)$$

$$= \arg \min_{\beta} [-\log p(\mathcal{D}|\beta) - \log p(\beta)] \quad (2.12)$$

The negative log likelihood term is the same as in the MLE, the prior must be defined. In a Bayesian approach to linear regression, the choice of the prior distribution $p(\beta)$ for the model parameters is determined by prior knowledge, desired model properties, and computational considerations. Most often, it is assumed that each of the parameters should not be large, with a mean of zero. This means that we do not have prior information about whether the parameters should be positive or negative and that we do not want the parameters to be too large in our model. This assumption works well when we need to control the parameters so that the model is not biased and does not have a high variance in its parameters. The particular choice of a prior distribution, globally, narrows down to two distributions: the Gaussian distribution with parameter $\mu = 0$ and the Laplace distribution with parameter $\mu = 0$. These distributions are symmetric and reflect different assumptions about the structure of the model parameters. A Gaussian a priori distribution with zero mean and fixed variance for the regression coefficients leads to L2-regularization (ridge regression). This reflects the belief that the model parameters are likely small but can be either positive or negative. It is also convenient analytically: combining a normal prior with a normal error model gives a normal posterior distribution, allowing a closed form for inference. The Laplace distribution with zero mean corresponds to the L1-regularization (lasso). The direct derivation of the L1 and L2 regularization terms according to Murphy (2022) from the a priori distributions is shown below.

In (Murphy, 2022, pp. 385-386) the L1 regularization is presented through the MAP estimation with Laplace prior. For L1 or lasso regularization the prior is defined to be a Laplace distribution with parameters $\mu = 0, b = \tau$:

$$p(\beta) = \prod_{i=1}^p \text{Laplace}(\beta_i|0,\tau) = \prod_{i=1}^p \frac{1}{2\tau} \exp\left(-\frac{|\beta_i|}{\tau}\right) \propto \prod_{i=1}^p \exp\left(-\frac{|\beta_i|}{\tau}\right), \quad (2.13)$$

where the normalization constant $\frac{1}{2\tau}$ can be ignored for minimization with respect to β . After substituting the negative log likelihood and Laplace prior from Eq. (2.13) into Eq. (2.12) and applying the logarithm the MAP estimation with Laplace prior yields:

$$\hat{\beta}_{MAP} = \arg \min_{\beta} \left[\sum_{i=1}^n (y_i - \beta^\top x_i)^2 + \lambda \sum_{i=1}^p |\beta_i| \right],$$

where $\lambda = \frac{1}{\tau}$ is the regularization hyperparameter, which controls the strength of regularization; see (Murphy, 2022, pp. 385-386)

In (Murphy, 2022, p. 381), similarly, the MAP estimation for L2 regularization is obtained by defining the prior as the Gaussian distribution with parameters $\mu = 0, \sigma^2 = \tau^2$:

$$p(\beta) = \mathcal{N}(\beta|0, \tau^2 I) \quad (2.14)$$

After substituting the Gaussian prior (Eq. (2.14)) and negative log likelihood (Eq. (2.6)) into Eq. (2.12) and performing mathematical transformation, we obtain the L2 regularized MAP estimation for parameters β :

$$\hat{\beta}_{MAP} = \arg \min_{\beta} \left[\sum_{i=1}^n (y_i - \beta^\top x_i)^2 + \lambda \sum_{i=1}^p \beta_i^2 \right],$$

where the λ parameter is defined as a λ proportional to the strength of the prior $\lambda = \frac{\sigma^2}{\tau^2}$, see (Murphy, 2022, p. 381)

Thus, the Bayesian MAP estimation for linear regression, under Gaussian or Laplacian priors, naturally introduces L2 or L1 regularization, respectively, connecting the Bayesian framework to penalized regression techniques in the frequentist paradigm.

Note that in linear regression, weights are related to the strength of the regressor's influence on the dependent variable. This is the key difference between models like linear regression or BPN and models like PNN and GRNN, where weights in the traditional sense do not exist, and the role of weights is played by the values of the kernel function or the distance between patterns. Thus, when adapting L1 and L2 regularization methods on PNN and GRNN models it is necessary to keep this in mind, because the regularization must be performed on weights, which are interpreted differently in models like PNN and GRNN.

3. Regularization in PNN and GRNN

In the context of Probabilistic Neural Networks, General Regression Neural Networks, and other kernel-based models that rely on kernel density estimation and distance metrics to make predictions, regularization is primarily achieved by reducing the influence of outliers and noisy data points, or by controlling the influence of the patterns based on its distance to the input vector, ensuring that the model generalizes well to unseen data.

Although regularization in PNN and GRNN has not been that well studied, certain effective methods exist to help reduce the effects of overfitting. In his work, Specht (1995) presents Adaptive PNN and Adaptive GRNN, which provide a mechanism to reduce the feature set or minimize the influence of less significant features. Experiments with these methods have shown their ability to improve model generalization by adapting the σ smoothing parameter for each pattern in the pattern layer, thus reducing the risk of overfitting. Zeinali and Story (2017) proposed another approach to enhance probabilistic neural networks called competitive probabilistic neural network (CPNN). Zeinali and Story (2017) suggested that not all Patterns from Pattern Layer should be used, but only those that are representative of a particular class and which are the most relevant for the input. Experiments show that CPNN outperforms traditional PNN in many cases, achieving higher classification accuracy by focusing on the most significant kernels, thus providing a practical and effective regularization strategy for these models. However, the methods listed above are specific to PNN and GRNN, and the focus of this work is to adapt the known and fundamental methods of regularization on the PNN and GRNN models.

3.1 Weights interpretation in PNN and GRNN

Weights (or parameters) in trainable models are generally associated with features. In models such as linear and logistic regression, weights can be interpreted as the influence strength of the features from the input vector. In BPN-like models, weights are associated with neurons in each layer that in turn perform the transformation of the input vector into the final response. Both of these interpretations have in common that they refer to feature space, in other words, they show the influence of a feature or an influence of the particular transformation of the feature vector. In the PNN and GRNN models, there are no weights in the direct sense. These models employ non-parametric estimation to make a prediction based on the statistical relationships encoded in the training set. Thus, the weights in these models could be interpreted as the influence of a particular object the training set (pattern) on the final response. Since these models are based on density estimation using KDE, it is the value of the kernel function from the input vector and the particular pattern that makes this contribution to the likelihood. The larger the kernel value, the greater the contribution of the pattern to the final estimate. Thus, to introduce regularization, we need to introduce a mechanism that will control the influence of each pattern to the final prediction.

3.2 Mixture of Gaussians

The PNN and GRNN employ non-parametric estimation of the underlying probability distributions using the KDE method. Although, in order to introduce the full Bayesian approach in PNN and GRNN estimation, the transition from non-parametric form to parametric must be done.

Consider a more general and simplified definition of the KDE shown in (Murphy, 2022, p. 561):

$$\hat{f}(x) = \frac{1}{n} \sum_{i=1}^n K(W_i, x),$$

where the kernel function $K(W_i, x)$ is typically a Gaussian kernel defined in Eq. (1.6). Note that this estimate can be interpreted as the average of Gaussian densities centered in W_i with the same variance σ^2 and equal weights $\frac{1}{n}$. This allows us to consider KDE in PNN and GRNN as a special case of the mixture of Gaussians models, which are discussed in (Bishop, 2006, pp. 110–113):

$$\hat{f}(x) = \sum_{i=1}^n b_i \mathcal{N}(x | W_i, \sigma^2 I),$$

where $b_i \in \beta$ are the component weights also called mixing coefficients, which in KDE are fixed as $b_i = \frac{1}{n}$, but can be made trainable. According to Bishop (2006, p. 111), the parameters β must meet the probabilities requirements.

$$\left(\sum_{i=1}^n b_i = 1 \right) \cap (\forall i : b_i \in [0, 1]). \quad (3.1)$$

The β are effectively weights that introduce the parametric weighting of the influence of the patterns W .

As mentioned in (Bishop, 2006, p. 113), all parameters in the mixture of Gaussians model are to be estimated, that is, the scale σ and the location parameter μ in each Gaussian distribution and parameters b_i . According to (Bishop, 2006, pp. 433–435), the parameters can be estimated with both the maximum likelihood and Bayesian approach. Moreover, the Bayesian approach to the estimation of these parameters prevents overfitting, which in most cases occurs when the scale parameters σ are estimated with maximum likelihood, which can lead to singularity as σ tends to zero. In the proposed PNN and GRNN models, we do not treat σ as random variables to be estimated, and location parameters μ are given by patterns from the train dataset. In order not to significantly misinterpret the classical PNN and GRNN, we will only estimate the parameters β , which have a straightforward interpretation.

However, it is worth noting that the adjustment of the σ parameter for each observation is also considered a powerful method, that can prevent overfitting. One of the methods proposed by Specht (1995) is adaptive GRNN and adaptive PNN.

3.2.1 Softmax transformation of a parameters

The parameters β , which will be optimized, must meet the requirements from Eq. (3.1). That is, they must be values from 0 to 1 and sum to 1. Gradient-based optimization methods, described in Sec. 3.3, do not require beta to be kept within these bounds, so for this purpose we will maximize the latent parameters β' and use the softmax function to obtain the correct parameters β before calculating the loss function:

$$\beta = \frac{\exp(\beta')}{\sum_{i=1}^n \exp(b'_i)} \quad (3.2)$$

This approach is inspired by the technique used by Bishop (1994) to transform the estimated parameters in mixture density networks.

3.2.2 Prior distribution definition

Introducing the prior distribution of b'_i will penalize the high influence of the points. The prior is purposely defined over the latent b'_i since the parameters b'_i do not have any constraints. The transformation to the β parameters is defined in Eq. (3.2). For L1 regularization, we define the prior using the Laplace distribution centered at zero and the scale parameter τ :

$$p_{L1}(b'_i) = \frac{1}{2\tau} \exp\left(-\frac{|b'_i|}{\tau}\right) \quad (3.3)$$

For L2 regularization, a Gaussian prior with center at zero and standard deviation (scale parameter) τ is used:

$$p_{L2}(b'_i) = \frac{1}{\sqrt{2\pi\tau^2}} \exp\left(-\frac{b'^2_i}{2\tau^2}\right) \quad (3.4)$$

3.2.3 General form of MAP estimation

Training of the parameters b'_i is achieved by maximizing the posterior probability of the parameters b'_i . Similarly, as detailed in (Murphy, 2022, sec. 4.5)

$$\hat{\beta}'_{MAP} = \arg \max_{\beta'} p(\beta' | y, X) \quad (3.5)$$

$$= \arg \max_{\beta'} [\log p(y|X, \beta) + \log p(\beta')] \quad (3.6)$$

$$= \arg \max_{\beta'} \left[\sum_{i=1}^n \log p(y_i | x_i, \beta) + \sum_{i=1}^n \log p(b'_i) \right], \quad (3.7)$$

where the β parameter is used in computing the likelihood and latent β' parameter used in the prior, see Sec. 3.2.1.

Since the β' parameters and the priors $p(b'_i)$ are the same for PNN and GRNN, we can extend this general formula for L1 and L2 regularized parameters for both GRNN and PNN.

Similarly, as described in Sec. 2.4 we substitute the term $\sum_{i=1}^n \log p(b'_i)$ with corresponding regularization term $-\frac{1}{\tau} \sum_{i=1}^n$ for the L1 regularization:

$$\hat{\beta}'_{MAP} = \arg \max_{\beta'} \left[\sum_{i=1}^n \log p(y_i|x_i, \beta) + \sum_{i=1}^n \log \left(\frac{1}{2\tau} \exp\left(-\frac{|b'_i|}{\tau}\right) \right) \right] \quad (3.8)$$

$$= \arg \max_{\beta'} \left[\sum_{i=1}^n \log p(y_i|x_i, \beta) - \frac{1}{\tau} \sum_{i=1}^n |b'_i| \right] \quad (3.9)$$

And, similarly, for L2 regularization we substitute the $p(b'_i)$ term in Eq. (3.7) with the prior defined in Eq. (3.4) and according to logic from Sec. 2.4 we substitute it with the L2 regularization term $-\frac{1}{2\tau^2} \sum_{i=1}^n b_i'^2$:

$$\hat{\beta}'_{MAP} = \arg \max_{\beta'} \left[\sum_{i=1}^n \log p(y_i|x_i, \beta) + \sum_{i=1}^n \log \frac{1}{\sqrt{2\pi\tau^2}} \exp\left(-\frac{b_i'^2}{2\tau^2}\right) \right] \quad (3.10)$$

$$= \arg \max_{\beta'} \left[\sum_{i=1}^n \log p(y_i|x_i, \beta) - \frac{1}{2\tau^2} \sum_{i=1}^n b_i'^2 \right] \quad (3.11)$$

The difference from the form of L2 regularization presented in (Murphy, 2022, Section 11.3) lies in the definition of the regularization coefficient λ . In the classical formulation, the regularization coefficient is given as $\lambda = \frac{\sigma^2}{\tau^2}$. In our case, the regularization coefficient becomes $\lambda = \frac{1}{2\tau^2}$, since σ is treated as a user-defined hyperparameter. It would therefore be inappropriate to make the regularization coefficient dependent on two separate user-specified parameters.

This is the general forms of the MAP β' estimation with L1 and L2 regularization. However, the term $p(y_i|x_i, \beta)$ is different for GRNN and PNN.

3.2.4 MAP estimation in PNN

The likelihood term $p(y_i|x_i, \beta)$ in the context of PNN is the probability of the input object, represented as input vector x_i assigned to class y_i . This can be represented with Bayes theorem as:

$$p(y_i|x_i, \beta) = \frac{p(x_i|y_i, \beta)P(y_i)}{\sum_{k=1}^K p(x_i|\theta_k, \beta)P(\theta_k)}, \quad (3.12)$$

where K is the number of possible classes and $\{\theta_1, \theta_2, \dots, \theta_K\}$ are unique classes of the classification problem. The term $p(x_i|y_i, \beta)$ is achieved with estimation using non-parametric KDE in the classic approach and with a mixture of Gaussians in the proposed approach. The prior probability of class $p(y_i)$ is known from the train data and K is the number of classes in the classification problem. In the proposed model, $p(x_i|y_i, \beta)$ is estimated using a mixture of Gaussians:

$$p(x_i|y_i, \beta) = \sum_{j=1, j \neq i}^{n_{y_i}} b_j^{y_i} \cdot \mathcal{N}(x_i|W_j^{y_i}, \sigma^2 I), \quad (3.13)$$

where $W_j^{y_i}$ represents the j^{th} pattern from the class y_i , n_{y_i} is the number of observations from class y_i in the training set and $b_j^{y_i}$ represents the parameter which corresponds to $W_j^{y_i}$. In addition, note that the estimation proceeds without the i^{th} pattern, for which we are trying to predict the probability of y_i given the input x_i . This is done because the likelihood for the i^{th} pattern will be too high because the Gaussian distribution will reach maxima at this point, which could result in an overly optimistic estimate of the predictive probability, as the model would essentially be evaluated at a point it was trained on, leading to overfitting. Substituting the estimate $p(x_i|y_i, \beta)$ in Eq. (3.13) into the likelihood in Eq. (3.12) gives the desired formula, which can be substituted into the MAP formula in Eq. (3.7):

$$p(y_i | x_j, \beta) = \frac{\sum_{j=1, j \neq i}^{n_{y_i}} b_j^{y_i} \cdot \mathcal{N}(x_i | W_j^{y_i}, \sigma^2 I) \cdot P(y_i)}{\sum_{k=1}^K \sum_{j=1, j \neq i}^{n_k} [b_j^{\theta_k} \cdot \mathcal{N}(x_i | W_j^{\theta_k}, \sigma^2 I)] \cdot P(\theta_k)} \quad (3.14)$$

Although at first glance the above notation may appear dense or unintuitive, a verbal explanation is provided below to help to understand the structure and interpretation of Eq. 3.14. The numerator summarizes all objects j belonging to the same class y_i as the considered object i (except the object i itself). For each such object, the contribution of the Gaussian density $\mathcal{N}(x_i, W_j^{y_i}, \sigma^2 I)$ weighted by the coefficient $b_j^{y_i}$ and the a prior probability of the class $P(y_i)$ is considered. Thus, the numerator reflects the unnormalized posterior probability that the object x_i belongs to the class y_i , given the data and model parameters. The denominator is summed in the same way, but for all classes θ_k and for all objects j in each of these classes (also excluding the object i). This normalizes the probability, taking into account the contribution of all classes and objects, and finally gives the correct estimated posterior.

The desired MAP formulation for PNN with L1 regularization is then:

$$\hat{\beta}'_{MAP} = \arg \max_{\beta'} \left[\sum_{i=1}^n \log \frac{\sum_{j=1, j \neq i}^{n_{y_i}} b_j^{y_i} \cdot \mathcal{N}(x_i | W_j^{y_i}, \sigma^2 I) \cdot P(y_i)}{\sum_{k=1}^K \sum_{j=1, j \neq i}^{n_k} [b_j^{\theta_k} \cdot \mathcal{N}(x_i | W_j^{\theta_k}, \sigma^2 I)] \cdot P(\theta_k)} - \frac{1}{\tau} \sum_{i=1}^n |b'_i| \right] \quad (3.15)$$

And MAP for PNN with L2 regularization is:

$$\hat{\beta}'_{MAP} = \arg \max_{\beta'} \left[\sum_{i=1}^n \log \frac{\sum_{j=1, j \neq i}^{n_{y_i}} b_j^{y_i} \cdot \mathcal{N}(x_i, W_j^{y_i}, \sigma^2 I) \cdot P(y_i)}{\sum_{k=1}^K \sum_{j=1, j \neq i}^{n_k} [b_j^{\theta_k} \cdot \mathcal{N}(x_i, W_j^{\theta_k}, \sigma^2 I)] \cdot P(\theta_k)} - \frac{1}{2\tau^2} \sum_{i=1}^n b_i'^2 \right] \quad (3.16)$$

3.2.5 MAP estimation in GRNN

The GRNN model employs the estimation of the joint probability density $f(x, y)$ to estimate the conditional mean. As defined in Eq. (1.2), the joint distribution in GRNN is estimated as:

$$\begin{aligned} \hat{f}(x, y) &= \frac{1}{(2\pi)^{(p+1)/2} \sigma^{(p+1)}} \cdot \frac{1}{n} \sum_{i=1}^n \exp \left(-\frac{(\|x - W_i\|_2^2)}{2\sigma^2} \right) \cdot \exp \left(-\frac{(y - y_i)^2}{2\sigma^2} \right) \\ &= \frac{1}{n} \sum_{i=1}^n \frac{1}{(2\pi)^{p/2} \sigma^p} \exp \left(-\frac{(\|x - W_i\|_2^2)}{2\sigma^2} \right) \cdot \frac{1}{\sqrt{2\pi}\sigma^2} \exp \left(-\frac{(y - y_i)^2}{2\sigma^2} \right) \end{aligned}$$

Note that the terms in the summation are the Gaussian probability densities $\mathcal{N}(x|W_i, \sigma^2 I)$ and $\mathcal{N}(y|y_i, \sigma^2)$. So, this formulation of the estimation of the joint probability density can be equally represented as follows:

$$\hat{f}(x, y) = \frac{1}{n} \sum_{i=1}^n \mathcal{N}(x|W_i, \sigma^2 I) \cdot \mathcal{N}(y|y_i, \sigma^2) \quad (3.17)$$

Following the same idea that the estimate $\hat{f}(x, y)$ can be considered as a special case of a mixture of Gaussians, where the parameters b_i are fixed as $\frac{1}{n}$, the equation (3.17) can be expressed as:

$$\hat{f}(x, y) = \sum_{i=1}^n b_i \cdot \mathcal{N}(x|W_i, \sigma^2 I) \cdot \mathcal{N}(y|y_i, \sigma^2) \quad (3.18)$$

To derive a MAP estimation for the GRNN, the likelihood $p(y_i|x_i, \beta)$ must be derived:

$$\begin{aligned} p(y_i|x_i, \beta) &= \frac{p(y_i, x_i|\beta)}{p(x_i|\beta)} \\ &= \frac{\sum_{j=1, j \neq i}^n b_j \cdot \mathcal{N}(x_i|W_j, \sigma^2 I) \cdot \mathcal{N}(y_i|y_j, \sigma^2)}{\sum_{j=1, j \neq i}^n b_j \cdot \mathcal{N}(x_i|W_j, \sigma^2 I)} \end{aligned}$$

Substituting this into equations Eq. (3.9) and Eq. (3.11) gives the desired MAP L1 and L2 regularized estimation for the GRNN model.

For L1 regularized MAP estimation is:

$$\hat{\beta}'_{MAP} = \arg \max_{\beta'} \left[\sum_{i=1}^n \log \frac{\sum_{j=1, j \neq i}^n b_j \cdot \mathcal{N}(x_i|W_j, \sigma^2 I) \cdot \mathcal{N}(y_i|y_j, \sigma^2)}{\sum_{j=1, j \neq i}^n b_j \cdot \mathcal{N}(x_i|W_j, \sigma^2 I)} - \frac{1}{\tau} \sum_{i=1}^n |b'_i| \right] \quad (3.19)$$

And for L2 regularized MAP estimation is:

$$\hat{\beta}'_{MAP} = \arg \max_{\beta'} \left[\sum_{i=1}^n \log \frac{\sum_{j=1, j \neq i}^n b_j \cdot \mathcal{N}(x_i|W_j, \sigma^2 I) \cdot \mathcal{N}(y_i|y_j, \sigma^2)}{\sum_{j=1, j \neq i}^n b_j \cdot \mathcal{N}(x_i|W_j, \sigma^2 I)} - \frac{1}{2\tau^2} \sum_{i=1}^n b_i'^2 \right] \quad (3.20)$$

3.2.6 Hyperparameter τ

The hyperparameter τ controls the strength of L1 and L2 regularization in the MAP estimation for the PNN and GRNN models. It determines how strongly the model penalizes large values of the weights b'_i . The lower the value of τ , the stronger the regularization, the higher the penalty for large weights. And vice versa, larger τ makes the regularization weaker, allowing the model to adjust more flexibly to the data. Similarly to the λ parameter in classical L1 and L2 regularization, defined in Sec. 2.4, τ adjusts the balance between the accuracy on training data and the generalizability of the model.

3.3 Analytical intractability and optimization strategy

The expression for MAP parameter estimation obtained in the previous section involves the logarithm of the ratio of the weighted sums of Gaussian functions in which the weights β themselves are the optimization parameters. This structure of the function leads to the non-analyticity of the target expression.

The reason is that the logarithm of the sum of exponents with parametric coefficients does not admit a closed-form analytical solution for the maximum. Moreover, the ratio of two such sums makes the problem fractionally rational in terms of parameters, which excludes the possibility of finding the extremum by differentiation and algebraic solution of the equation.

3.3.1 Gradient Descend

The only group of methods that allows to estimate a maxima of posterior probability is gradient methods. The gradient descend method employs numerical computation of a gradient. The gradient is a vector of partial derivatives of differentiable scalar function $f(\beta')$ at point $\beta' = [b'_1, b'_2, \dots, b'_n]$ (Goodfellow et al., 2016):

$$\nabla f(\beta' = [b'_1, b'_2, \dots, b'_n]) = \begin{bmatrix} \frac{\partial f}{\partial b'_1}(\beta') \\ \frac{\partial f}{\partial b'_2}(\beta') \\ \vdots \\ \frac{\partial f}{\partial b'_n}(\beta') \end{bmatrix} \quad (3.21)$$

The gradient gives the direction of the largest growth of the function at this point. i^{th} element in gradient vector $\frac{\partial f}{\partial b'_i}(\beta')$ indicates how fast the function f grows when the variable b_i increases and all other things are equal (other variables are fixed). The gradient descend method searches for the minimum of a function by iteratively changing the input variables β' by $-\gamma \nabla f(\beta')$ until the function converges to a minimum (Goodfellow et al., 2016):

$$\beta'_{n+1} = \beta'_n - \gamma \nabla f(\beta'),$$

where γ is a learning rate, which controls how fast the variables β' will be adjusted and n is a number of iterations. (Goodfellow et al., 2016) This process continues until the function converges to a local minimum. However, convergence is not exact and is usually controlled by the tolerance level. The tolerance defines how small the change in the function value or in the parameters must be between iterations for the algorithm to stop. (Goodfellow et al., 2016)

In the context of the proposed GRNN and PNN trainable model with trainable parameters β' , the function f is called a loss function. We defined a function of posterior probability in Eq. (3.15), (3.16), (3.19), (3.20), which is to be maximized. However, since the loss function must be minimized, we minimize the negative of the posterior probability functions.

3.3.2 Adam

Adam is another popular and widely used gradient optimization method, proposed by Kingma and Ba (2017). This method serves as an excellent general-purpose method that is suitable for many problems, especially when it comes to optimization, where sparse gradients may be encountered. Also, the gradient descend algorithm is very sensitive to the learning rate, since this is the only term that controls how much the parameters will change in iteration, and Adam is less sensitive to it. This is achieved using biased first moment estimates and biased second moment estimates of the gradients. The Adam algorithm for optimizing the parameters β is presented in the Appendices, Algorithm 1.

This algorithm is used in optimizing the parameters in the proposed models because of its stability and flexibility.

4. Demonstration of regularization in PNN and GRNN on synthetic data

To illustrate the behavior of L1 and L2 regularized MAP estimation introduced in previous sections, we construct a simple two-dimensional synthetic dataset. This setting allows for an intuitive visualization of the β parameters.

Note. Further in this work, for the implementation of PNN and GRNN models, my own code, available on GitHub (Vinogradov, 2025), is used. Also, the code for the outputs presented is available. Synthetic dataset generation and pre-processing are performed using the functionality provided by the Scikit-learn project (Pedregosa et al., 2011). In addition, further in this work, base GRNN and PNN models employ Gaussian kernel.

4.1 Data

To support the empirical demonstration of the regularization techniques described earlier, we construct a synthetic classification dataset using `make_classification` function from a scikit-learn Python library. This dataset includes two informative features and two well-separated classes, allowing a clear visual analysis of the effect of L1 and L2 regularization on model behavior. A portion of the resulting training set is shown in Table 4.1, where each row corresponds to a training pattern with its respective class label, which is a target variable.

	Feature 1	Feature 2	Class
0	-1	5	1
1	-5	-5	1
2	-17	-14	0
3	-10	-8	0
4	-4	-2	0
5	4	-1	1
6	0	1	0
7	-6	-5	0
8	-9	-4	0
9	17	3	1
10	-5	-1	0
11	-19	-19	0
12	-1	-2	1
13	-3	-8	1
14	12	-11	1

Table 4.1: Demonstration synthetic dataset.

Table 4.1 illustrates the spatial distribution of the training samples for both classes and highlights the location of the input vector.

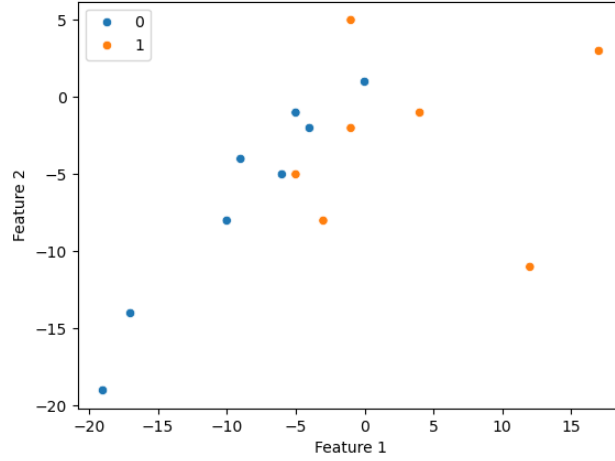


Figure 4.1: Visualization of the training samples and the input vector

4.2 Models

The experiment employs a consistent training dataset, with the only variation being the type of regularization applied. The pattern layer uses a Gaussian kernel with $\sigma = 3$ and $\tau = 3$. The

choice of hyperparameters was made only to make the visualization and demonstration of the regularization effect clearer and was not made for better classification. During the experiment, 3 models were initialized and trained: a regular PNN, a PNN with L1 regularization, and a PNN with L2 regularization. The models use the same hyperparameters and the same dataset.

4.3 Results of parameter estimation

Table 4.2 summarizes the result of L1 and L2 regularized MAP estimation of β parameters. For each training sample, the table includes its feature values, initialized β' and β parameters, L1 and L2 regularized MAP estimated parameters.

	Pattern	Init. b'_i	Init. b_i	L1 est. b'_i	L1 est. b_i	L2 est. b'_i	L2 est. b_i	Class
0	[-1; 5]	-0,002	0,040	-0,000	0,060	-0,261	0,052	1
1	[-5; -5]	-0,284	0,030	0,000	0,060	-1,518	0,015	1
2	[-17; -14]	-0,279	0,030	-0,000	0,060	0,026	0,069	0
3	[-10; -8]	-2,099	0,005	-1,092	0,020	-0,522	0,040	0
4	[-4; -2]	0,927	0,101	0,022	0,061	-0,479	0,042	0
5	[4; -1]	-2,703	0,003	-1,694	0,011	0,767	0,144	1
6	[0; 1]	0,551	0,070	-0,144	0,052	-1,409	0,016	0
7	[-6; -5]	2,094	0,326	1,224	0,204	0,032	0,069	0
8	[-9; -4]	-1,059	0,014	-0,012	0,059	0,062	0,071	0
9	[17; 3]	-1,568	0,008	-0,569	0,034	1,026	0,187	1
10	[-5; -1]	0,250	0,052	-0,000	0,060	-0,329	0,048	0
11	[-19; -19]	0,993	0,108	-0,003	0,060	0,024	0,069	0
12	[-1; -2]	-0,346	0,028	0,221	0,075	0,074	0,072	1
13	[-3; -8]	0,928	0,102	0,727	0,124	-0,804	0,030	1
14	[12; -11]	0,718	0,082	0,000	0,060	0,119	0,076	1

Table 4.2: Summary of the L1 and L2 regularization outcomes.

Source and processing: author

To visually illustrate this effect, refer to table 4.1, which displays the patterns and the input vector. To observe the impact of L1 and L2 regularization on the graph, we resize the patterns according to their b_i parameter value:

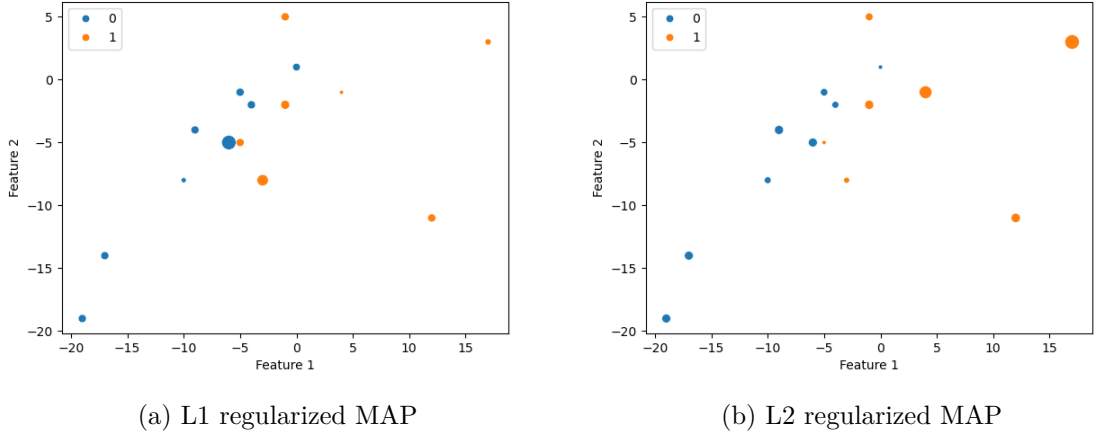


Figure 4.2: Effect of regularization type on training pattern's b_i parameter.

Source and processing: author

The visualization in Figure 4.2 shows how the use of different types of regularization affects the relative importance of training patterns. It demonstrates that L1 regularized MAP estimation provide a local effect, while the influence of L2 regularized MAP estimation can be described as distributed across all patterns.

4.3.1 Estimated density shape

To demonstrate the effect of L1 and L2 regularization, below is a visualization of the estimation of the probability density for the feature vector $\hat{f}(x)$ with KDE (classical PNN) 4.3, and with a mixture of Gaussians with L1 4.4 and L2 4.5 regularized MAP estimation for parameters.

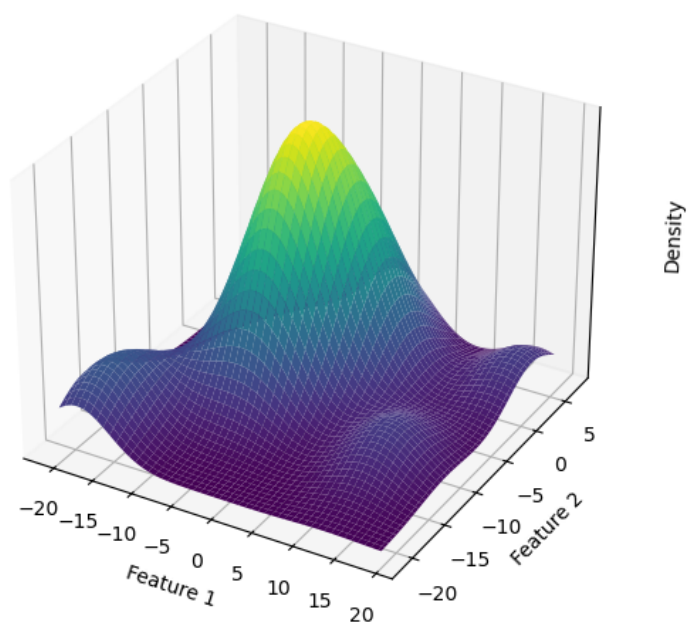


Figure 4.3: Estimated density distribution with KDE (Gaussian kernel).

Source and processing: author

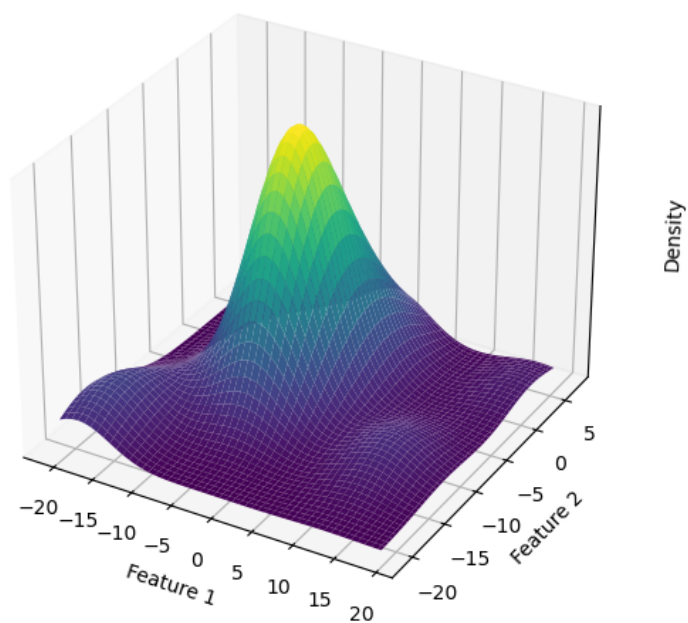


Figure 4.4: Estimated density distribution with mixture of Gaussians (L1 regularized MAP).

Source and processing: author

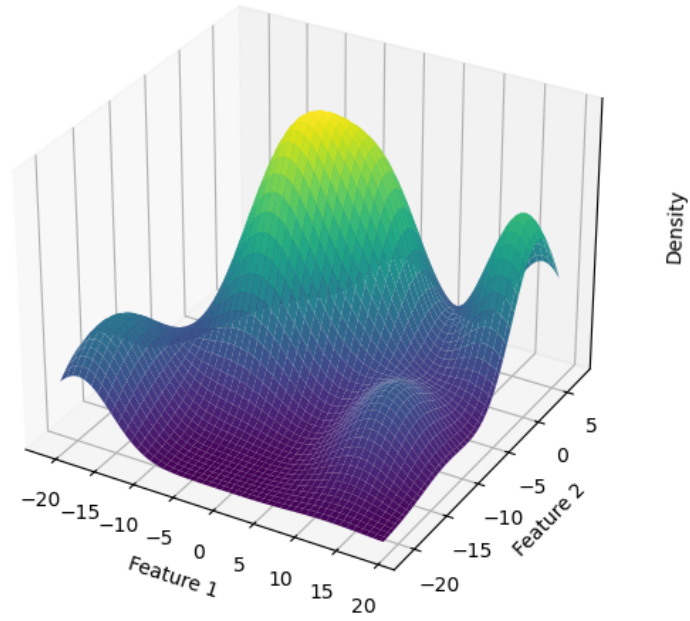


Figure 4.5: Estimated density distribution with mixture of Gaussians (L2 regularized MAP).

Source and processing: author

The figures show the effect of regularization on the estimation of the distribution density $f(x)$. As can be seen in Fig. 4.4, L1 regularization has the effect of making the surface sharper and more compressed. This is due to the very nature of the L1 regularization, which is stricter and makes larger differences in the parameters, focusing on the more useful ones and ignoring more the uninformative ones. On the other hand, Fig. 4.5 shows that regularizing L2 provides more balanced parameters, which are explained by a milder penalty.

4.3.2 Decision boundaries

The figures below visualize the solution bounds for the classic PNN model, the model with trained parameters, and L1 regularization and L2 regularization. A decision boundary is a line (in 2D) or a hyperplane (in higher dimensions) that divides the feature space into regions corresponding to different classes. That is, at any point in the space, the model will predict one of the classes depending on which side of the boundary this point is located.

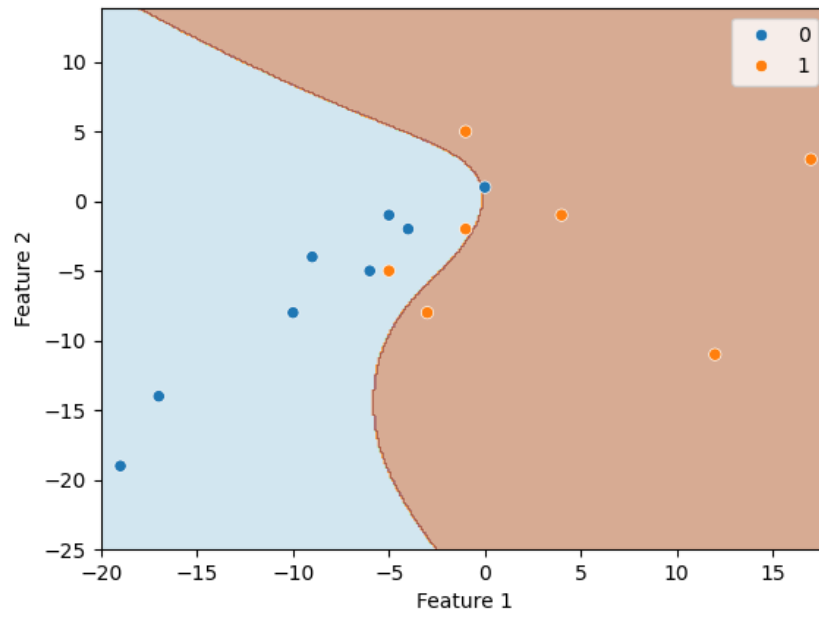


Figure 4.6: Decision boundary for classification (classic PNN).

Source and processing: author

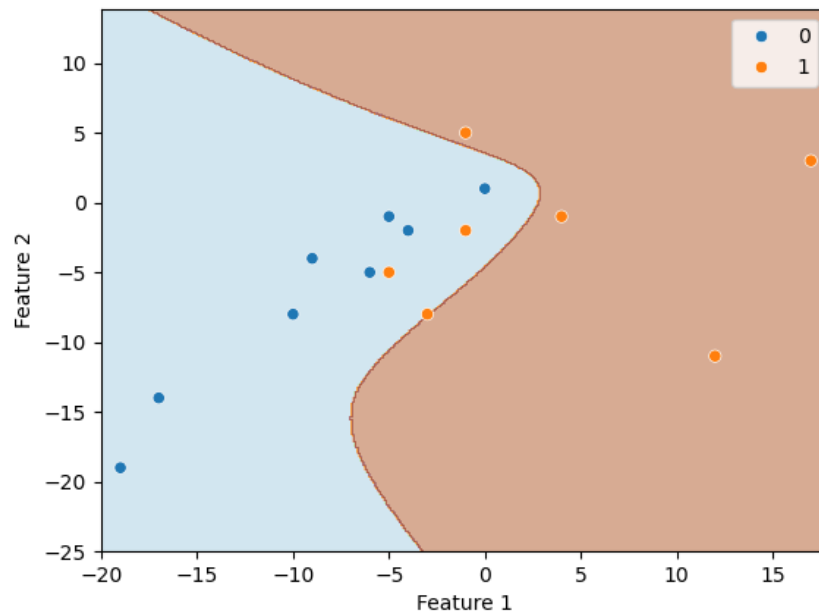


Figure 4.7: Decision boundary for classification (L1 regularized MAP).

Source and processing: author

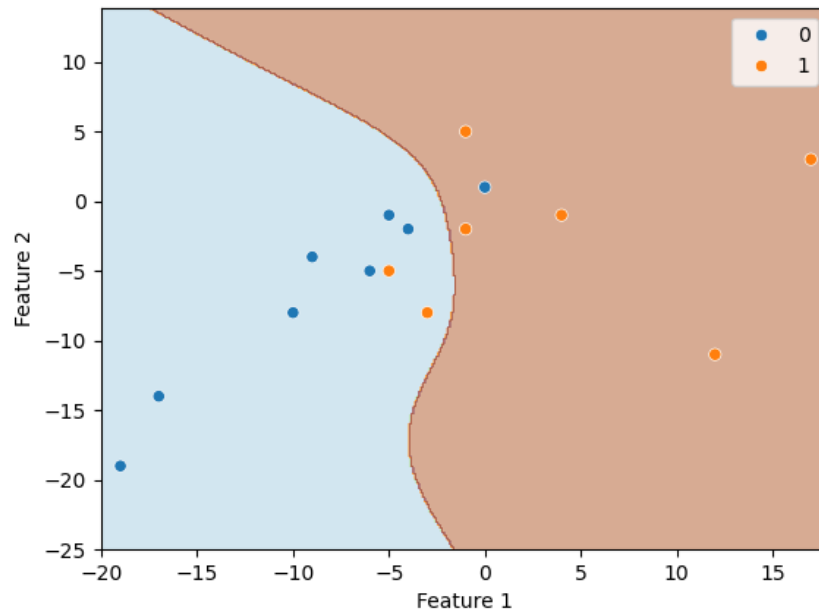


Figure 4.8: Decision boundary for classification (L2 regularized MAP).

Source and processing: author

Figures 4.6, 4.7, 4.8 show the difference in the decision boundaries. You can see how the decision boundaries follow the density shapes described in Figures 4.3, 4.4, 4.5. As can be seen in Fig. 4.6, the classic PNN makes the decision boundary quite smooth. Fig. 4.7 shows the decision boundary for the model using L1 regularized MAP estimator. Here we can see how the weights adjusts more well to the shape of class 0 by capturing one of the patterns, however, the boundary is not very smooth. Fig. 4.8 shows the decision boundary for the model using L2 regularized MAP estimator. Here we can observe a smoother boundary.

4.3.3 Weights values comparison

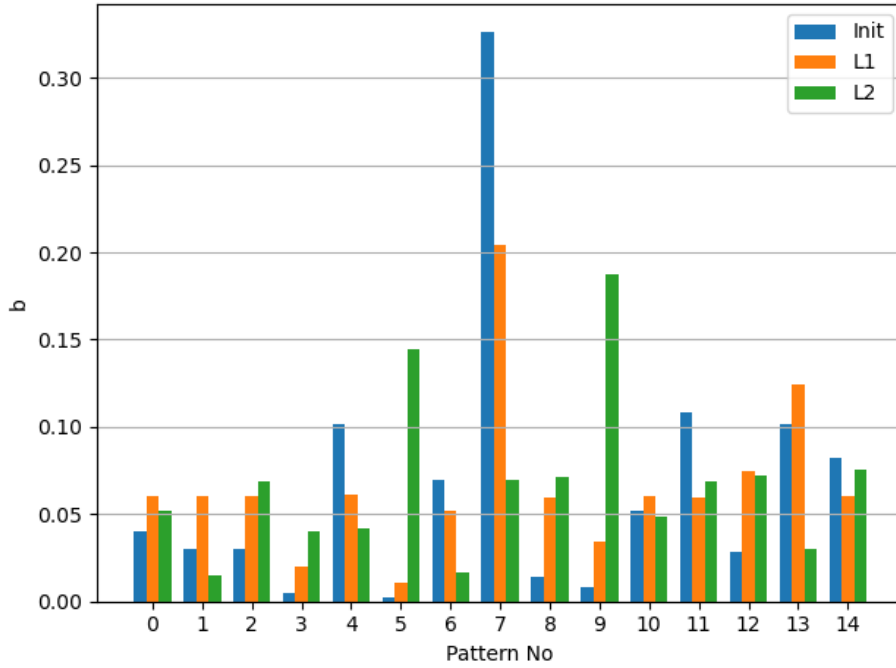


Figure 4.9: Comparison of initial, L1 and L2 estimated weights.

Source and processing: author

The Fig. 4.9 depicts the values, that were estimated during L1 and L2 regularized MAP. As can be seen in Fig. 4.9, L2 regularization more often gives bigger values or more balanced values than L1 regularization. The same effect can be seen in Fig. 4.2. It can also be observed how the estimation is able to adjust to MAP even if the initial values for the β' parameters were too large, this can be clearly seen for patterns 7 and 9.

5. Evaluation on real datasets

5.1 Preprocessing, metrics and model selection

For evaluation, the metrics Mean Squared Error (MSE) and Accuracy were chosen to evaluate the performance of the GRNN and PNN models respectively. Further, the percentage improvement for each regularized model compared to the classic models was calculated. The effectiveness was assessed as follows:

1. Find the best parameter σ for the classical PNN and GRNN models using 4-fold cross validation;
2. For the same parameter σ find the best τ for L1 and L2 regularization models using 4-fold cross validation.

The following ranges of hyperparameters were selected for model tuning:

- σ : from 0,1 to 5,0 in steps of 0,1
- τ : 0,1 to 5,0 in steps of 0,1

Both ranges were generated using a uniform step size.

For data preprocessing, only standard scaling was used. The scaler retrieved information to perform scaling exclusively on the training split within each iteration of the 4-fold cross validation. The scaling was then applied to the corresponding test split. This avoided data leakage between the training and test samples.

In regression problems, both the feature data and the target variable were scaled. In order to correctly interpret the quality metric, the inverse transformation of predictions so than the MSE was calculated at the original scale.

In case there were missing values in the rows of the original dataset, the corresponding rows were completely removed from the sample before training the model.

The hyperparameters β_1 and β_2 for Adam optimization algorithm were chosen by recommendation from (Kingma & Ba, 2017), $\beta_1 = 0,9$, $\beta_2 = 0,999$. The learning rate was chosen to be $\gamma = 0,001$.

5.2 Classification

PNN models were evaluated with binary classification tasks on the following datasets:

1. The Breast Cancer dataset (Wolberg et al., 1993), where the goal is to predict whether a patient has breast cancer. Features include measurements of cell nuclei such as ra-

dus, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension.

2. The Suspicious Firms dataset from audit data (Hooda, 2018), aiming to classify firms as fraudulent or not. Features include financial ratios, audit indicators, and firm characteristics derived from audit records.
3. The Parkinson’s dataset (Little, 2007), used to identify Parkinson’s disease presence from quantitative voice measures. The features consist of various acoustic measures, including jitter, shimmer and other measures extracted from voice recordings.

Table 5.1 illustrates the outcome of models tuning and evaluation for classification tasks:

Dataset	σ	Acc. Base	τ (L1)	Acc. L1	Imp. % (L1)	τ (L2)	Acc. L2	Imp. % (L2)	Instances	Logistic Regression acc.
Breast Cancer ¹	0,8	0,965	2,6	0,968	0,364	4,3	0,970	0,549	569	0,977
Suspicious firms ²	0,4	0,955	0,2	0,916	-4,050	0,9	0,916	-4,050	775	0,981
Parkinson’s ³	1,6	0,804	2,3	0,840	4,442	4,3	0,840	4,442	195	0,789

Table 5.1: Models evaluation on selected datasets for classification problem

Source and processing: author

Datasets sources: (Hooda, 2018; Little, 2007; Wolberg et al., 1993)

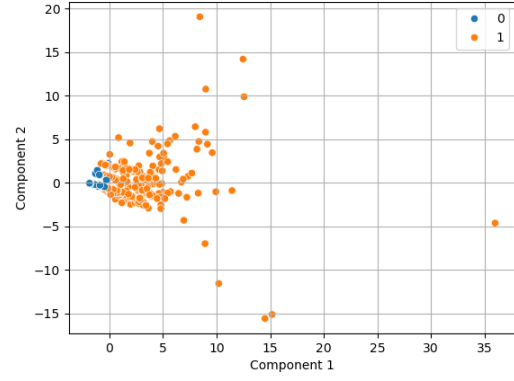
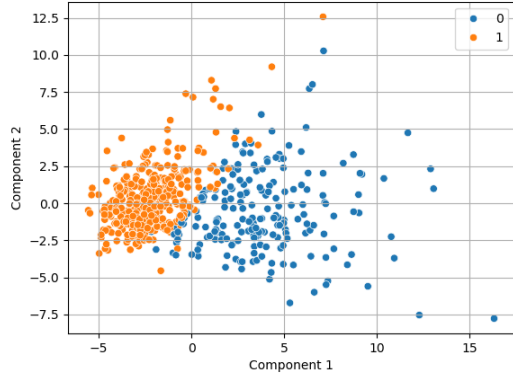
As shown in the Table 5.1, L1 and L2 regularization in 2 of 3 datasets showed an improvement in accuracy. However, in the Suspicious firms dataset, there was no improvement in prediction accuracy in the models with L1 and L2 regularization and we observe an accuracy decrease of 4,050%. In the Breast Cancer and Parkinson’s datasets we see an improvement between 0,36% and 4,44%. Also in the table is a comparison with the classical Logistic Regression model, which showed better performance on the Breast Cancer and Suspicious firms datasets, but loses in accuracy on the Parkinson’s dataset. This indicates that the data were well linearly separated in the Breast Cancer and Suspicious firms, due to which logistic regression showed higher accuracy.

In order to test why logistic regression gets better accuracy in some datasets, we will perform a principal component analysis (PCA) to reduce dimensionality into two components and see how the classes fit in the new feature space. As described in (Murphy, 2022, p. 657), PCA is the simplest and most widely used form of dimensionality reduction that finds a linear projection of high-dimensional data onto a low-dimensional subspace, preserving the structure of the data. This will allow us to visually assess how well the data separate into classes in the low-dimensional representation and understand how linearly separable the classes are. If the classes appear to be clearly separable by the first two principal components, this would explain the high performance of logistic regression.

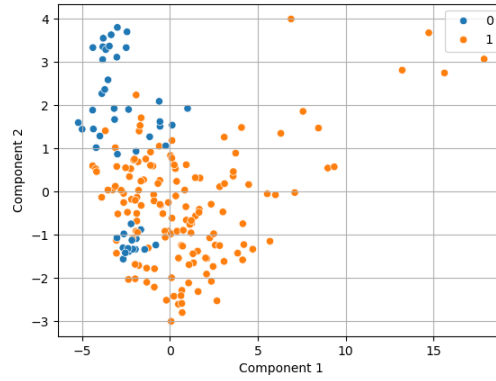
¹Breast Cancer dataset (Wolberg et al., 1993): classification task to predict breast cancer.

²Suspicious firms dataset (Hooda, 2018): classification task to detect fraudulent firms.

³Parkinson’s dataset (Little, 2007): classification task to detect Parkinson’s disease from voice data.



(a) PCA visualization for Breast Cancer dataset (b) PCA visualization for Suspicious firms dataset



(c) PCA visualization for Parkinson's dataset

Figure 5.1: Class Separation in 2D PCA Space

Source and processing: author

Datasets sources: (Hooda, 2018; Little, 2007; Wolberg et al., 1993)

As can be seen in Fig. 5.1, for the Breast Cancer and Suspicious firms datasets the classes are well linearly separated, the two classes form distinct clusters that are easily divided by a linear boundary, which is ideal for logistic regression. In contrast, the Parkinson's dataset does not demonstrate such a clear linear separation. Although class 1 forms a large solid cluster, class 0 appears to be fragmented into two distinct subclusters. This structure suggests that a linear model may struggle to capture the true decision boundary effectively. To investigate this further, we visualize the decision boundaries learned by both the PNN with L2 regularization and the classical logistic regression model.

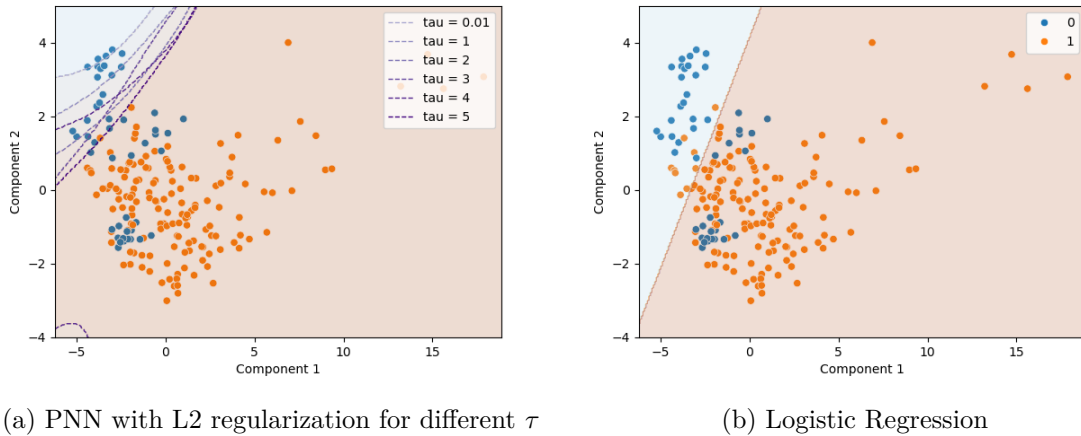


Figure 5.2: Decision boundaries in 2D PCA Space for Parkinson's dataset

Source and processing: author

Dataset source: (Little, 2007)

Fig. 5.2 demonstrate the decision boundaries for PNN with L2 regularization and logistic regression for Parkinson's dataset. The PNN model (left) shows a more flexible, non-linear boundary, better adapting to the structure of the data, whereas logistic regression (right) constructs a strictly linear boundary without considering complex class distributions. Also, the figure on the left shows how the τ parameter affects the decision boundaries. For very small values of the parameter τ (that is, strong regularization), the decision boundaries generated by PNN become irrelevant and cannot adequately reflect the underlying data structure. But as soon as regularization is reduced, the model begins to adjust to the data and to construct the boundary well.

In addition, it is worth noting that within the framework of the PNN model, capturing a second cluster of class 0 can theoretically be achieved by applying an adaptive PNN (Specht, 1995), for example. Thus, PNN is useful when the data are not linearly divided into classes but there are separated clusters of classes.

5.3 Regression

For GRNN models, the following datasets were used for regression tasks:

1. The Diabetes dataset (Efron et al., 2004), used to estimate a quantitative metric for disease progression, measured as a progression score. The features include age, sex, body mass index, average blood pressure, and six blood serum measurements.
2. The Concrete Compressive Strength dataset (Yeh, 1998), involving estimation of the compressive strength of concrete, measured in megapascals (MPa). The features are

the proportions of cement, blast furnace slag, fly ash, water, superplasticizer, coarse aggregate, and fine aggregate, as well as the age of the concrete.

3. The Energy Efficiency dataset (Tsanas & Xifara, 2012), where the heating load of a building is used as the target variable, measured in kilowatts (kW). Features cover the geometry of the building and the characteristics of the glazing.

Table 5.2 illustrates the outcome of models tuning and evaluation for regression task:

Dataset	σ	MSE Base	τ (L1)	MSE L1	Imp. % (L1)	τ (L2)	MSE L2	Imp. % (L2)	Instances	Linear Regression MSE
Diabetes ¹	1,0	3251,300	4,6	3156,055	3,018	2,9	3125,046	4,040	442	2961,710
Concrete										
Compressive Strength ²	0,7	165,845	2,3	160,626	3,249	3,3	159,922	3,703	1030	137,018
Energy Efficiency ³	0,2	7,486	2,0	7,516	-0,399	0,5	7,565	-1,048	768	10,113

Table 5.2: Models evaluation on selected datasets for regression problem

Source and processing: author

Datasets sources: (Efron et al., 2004; Tsanas & Xifara, 2012; Yeh, 1998)

As shown in Table 5.2, applying L1 and L2 regularization led to consistent performance improvements in 2 out of 3 datasets. In the Diabetes dataset, L1 reduced the MSE by 3,02%, while L2 achieved a larger reduction of 4,04%. For Concrete Strength, L1 provided a 3,25% improvement, while L2 gave 3,70% improvement. However, in the Energy Efficiency dataset, regularization did not result in improvement. In contrast, both L1 and L2 slightly increased MSE by 0,40% and 1,05%, respectively, indicating that regularization may not be beneficial for some data. Despite this, both regularized models and the base GRNN still outperformed standard Linear Regression, which had a significantly higher MSE of 10,113, compared to 7,516 (L1) and 7,565 (L2). Interestingly, the opposite was true for the other two datasets. In both Diabetes and Concrete Compressive Strength, while regularization improved performance relative to the base model, their MSE values remained higher than those achieved by standard linear regression. This indicates that, although regularization helps reduce overfitting in some scenarios, it may not always lead to better absolute performance when compared to a well-fitted linear model.

To better understand the structure of the data and the behavior of the models, we applied PCA. This approach captures the direction of maximum variance and allows for a simplified visualization. For each dataset, we projected all instances onto the first principal component, which was then plotted on the x-axis. The corresponding target values were plotted along the y-axis. In the same plot, we also included the predictions made by linear regression

¹Diabetes dataset (Efron et al., 2004): regression task to estimate the quantitative metric for disease progression.

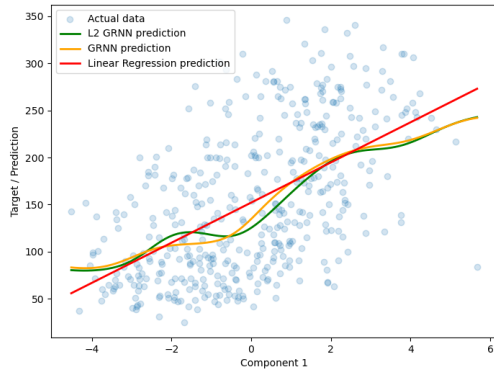
²Concrete Compressive Strength dataset (Yeh, 1998): regression task to estimate the compressive strength of concrete in megapascals.

³Energy Efficiency dataset (Tsanas & Xifara, 2012): regression task to predict the heating load of a building (kW).

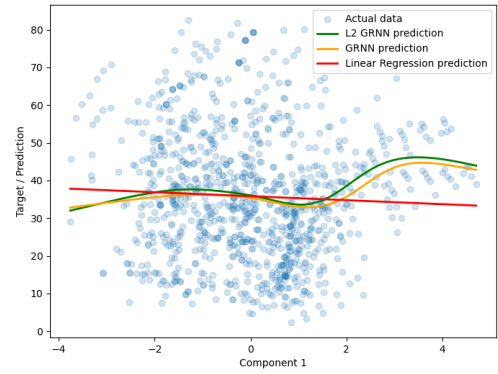
base GRNN, and L2 regularized GRNN with hyperparameters according to Table 5.2. Similarly, the scaling was performed for both features and target, the inverse transformation was performed only for target.

As shown in Fig. 5.3, the most noticeable effect of L2 regularization is observed in the Diabetes dataset, where the regularized GRNN model demonstrates visibly smoother and more refined predictions. In contrast, for the other datasets, there is no clear visual difference between the regularized and unregularized GRNN predictions.

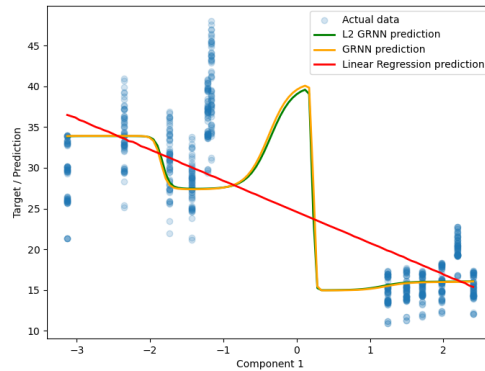
Another insight drawn from the visualization is that the Energy Efficiency dataset exhibits a predominantly non-linear relationship between the features and the target variable. This observation explains why the GRNN models, which are capable of capturing non-linear patterns, outperform linear regression on this particular dataset.



(a) PCA visualization for Diabetes dataset



(b) PCA visualization for Concrete Compressive Strength dataset



(c) PCA visualization for Energy Efficiency dataset

Figure 5.3: Regression visualization in 2D PCA Space

Source and processing: author

Datasets sources: (Efron et al., 2004; Tsanas & Xifara, 2012; Yeh, 1998)

5.4 Learning curve

To analyze the behavior of the generalization ability of regularized GRNN and PNN models depending on the training sample size, we construct a learning curve. The learning curve is a plot of the dependence of error (e.g., mean square error) on the training and/or test sample on the number of training examples. Such analysis allows us to reveal the nature of overfitting, to determine at what amount of data the model achieves stable quality, and to compare the effectiveness of different regularization methods.

The Diabetes dataset was selected in order to plot the learning curve.

1. Training and test set were sampled, where 70% of the data was allocated to training and 30% to testing;
2. To construct the learning curve, the training sample was additionally selected in fractions of 10% up to 90% in steps of 10% of the training set from previous set;
3. For each fraction of training set, the models evaluated using MSE metric on the test set from first step.

The GRNN hyperparameters were selected according to Table 5.2.

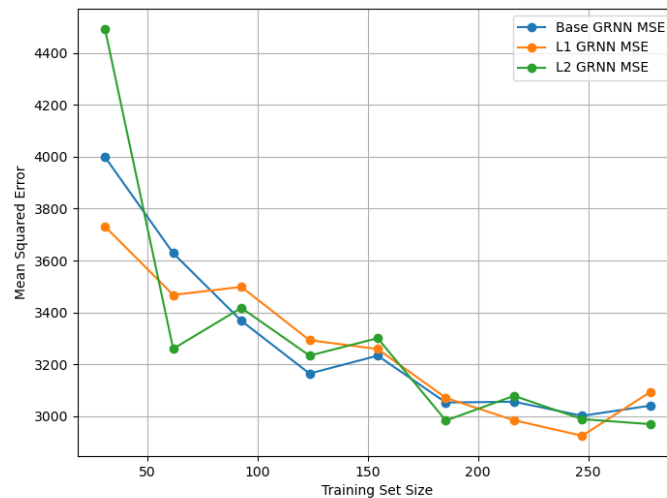


Figure 5.4: Learning curve for Diabetes dataset.

Source and processing: author

Dataset sources: (Efron et al., 2004)

Figure 5.4 demonstrates that for smaller training set sizes (up to 100 samples), the models with L1 and L2 regularization tend to produce more stable and generally better performance (except L2 regularized model for the smallest train set). However, as the training data increases, all models gradually converge to similar performance levels. There are also instances

where the regularized models perform worse than the base GRNN, which can be attributed to the randomness in the train-test split and the stochastic initialization of the β' parameters in the GRNN. The experiment was conducted using a fixed random seed of 42.

Similarly, we conducted an experiment with the same settings, but with Concrete Compressive Strength data set. This dataset has more observations, that is, 1030 instances. The result is presented in Fig. 5.5.

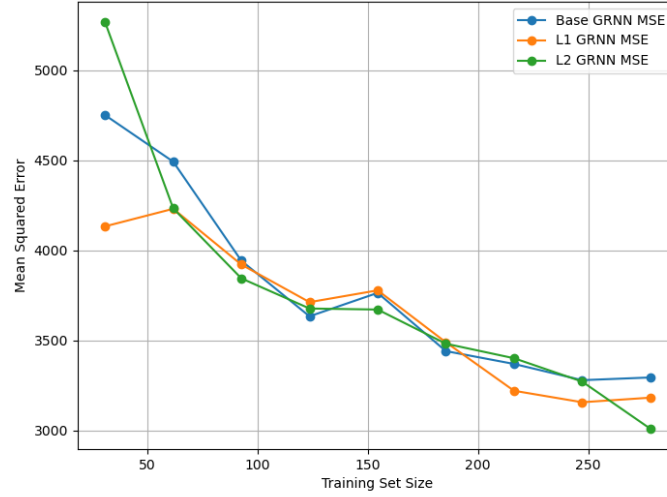


Figure 5.5: Learning curve for Concrete Compressive Strength dataset.

Source and processing: author

Dataset sources: (Yeh, 1998)

This experiment shows almost the same results.

In general, there is no clear evidence that the performance of the model is strongly dependent on the size of the training set. This observation is supported not only by the learning curve experiment, but also by evaluations of the models across different datasets. The only conclusion that can be drawn from the observations is that, on very small training sets, regularization can reduce the error.

5.5 The effect of regularization strength on model error

This section investigates the effect of the regularization parameter on the performance of the GRNN model. We investigate how L1 and L2 regularization and the value of the parameter τ affect the model error on both the training and test samples.

The following experiment was performed using the Diabetes dataset. To analyze the effect

of regularization strength on the quality of the GRNN model, a series of evaluations was carried out with the regularization parameter τ varying in the range from 0.1 to 5.0 with a step of 0.1. For each value of τ , models with two types of regularization, L1 and L2, were trained. Training and test data were split, 30% of the data was allocated to the test sample, and standardized for features and target variable.

Each model was trained on scaled training data, and predictions were built for the training and test samples. The resulting predictions were inversely scaled to the original scale of the target variable. The MSE was used to assess quality.

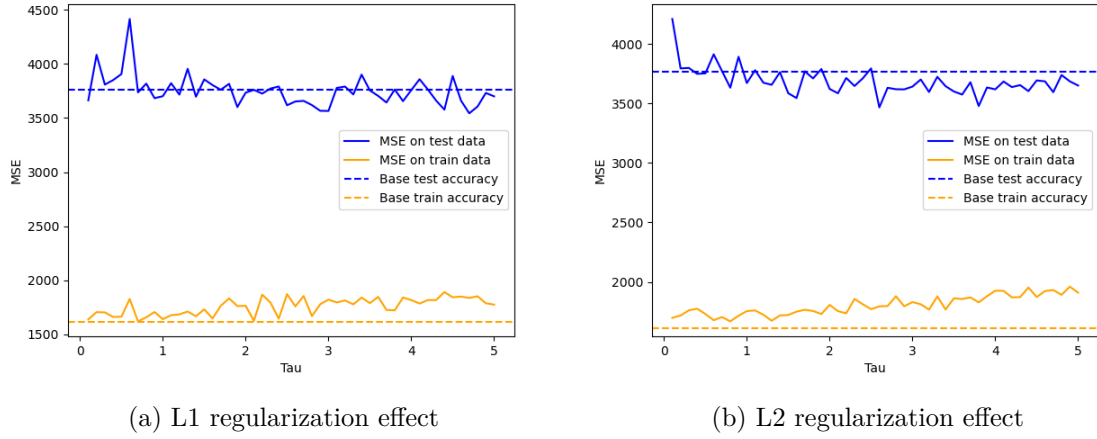


Figure 5.6: The effect of regularization strength on MSE

Source and processing: author

Dataset sources: (Efron et al., 2004)

Fig. 5.6 illustrates the dependence of MSE on the regularization parameter τ . As can be seen, the gap between training and test MSE is generally significant, indicating overfitting. However, as the regularization strength approaches the optimal values, this gap narrows. This suggests that regularization has a positive effect on reducing overfitting and improving the generalization performance of the model.

Conclusion

This bachelor thesis was dedicated to the study and implementation of regularization methods in non-parametric models of general regression neural networks and probabilistic neural networks. The main objective of this bachelor thesis was to investigate the known regularization methods, apply them on GRNN and PNN, implement them in a programming language, and evaluate the effectiveness of these methods on real data.

Within the theoretical part, the mathematical and statistical foundations that underlie the PNN and GRNN models have been described in detail. These include the principles of Bayesian optimal decision, general regression, distance metrics, and kernel density estimation. The reasoning and foundation of regularization, what regularization is and what overfitting is and how to introduce L1 and L2 regularization based on Bayesian approach were further investigated. Thus, the first aim of this work is fulfilled.

The practical part of the work consisted of the sequential development of the GRNN and PNN models that implement L1 and L2 regularization. The base models of GRNN and PNN were adapted to the idea of a mixture of Gaussians model in probability estimation, instead of KDE as it is realized in the base models. A form of MAP estimation for the parameters of the mixture of Gaussians model within PNN and GRNN was also derived, within which L1 and L2 regularized parameter estimates were obtained by introducing a prior distributions in the parameters. Then these models, including the basic GRNN and PNN models, were implemented in the Python programming language. The implementation is available through the GitHub repository (Vinogradov, 2025). An illustration of the work was done on a synthetic dataset, where the main effect of L1 and L2 regularization is underlined in the GRNN and PNN models. The performance of the proposed methods has been evaluated on a set of real data using 4-fold cross-validation. Therefore, the implementation and evaluation goals were fully met.

Experiments confirmed the practical benefits of applying L1 and L2 regularization to non-parametric neural network models such as GRNN and PNN on some data. In classification tasks, the effect of regularization varied across datasets: it improved performance on Parkinsons dataset, while it showed limited or even negative effects on other datasets, which are Suspicious Firms and Breast Cancer. Similarly, in regression tasks, regularization improved performance in some cases, such as in the Concrete Compressive Strength and Diabetes datasets, where MSE was reduced, while in the Energy Efficiency dataset performance slightly decreased.

Visualizations using PCA and decision boundary plots clarified why non-linear models often outperform traditional methods like logistic regression in complex data distributions and why linear models sometimes outperform more sophisticated ones. Specifically, regularized PNN excelled at capturing fragmented or non-linear class structures of Parkinsons dataset, demonstrating their strength in such scenarios. Learning curve analysis on the Diabetes

dataset revealed that regularization boosts generalization, particularly with smaller training sets. This indicates that regularization is most effective when the amount of data is limited or the data structure is complex.

The evaluation of the models on real data showed mixed results. On the one hand, on some datasets the proposed methods improved the accuracy of predictions, on the other hand, on others their efficiency was lower compared to the basic approaches. This variability emphasizes the importance of taking into account the data specifics when applying regularization methods to GRNN and PNN models. It could also be concluded that the proposed method could show an improvement in performance on only some data.

One of the main limitations encountered in this research was the substantial amount of time and computational resources required for hyperparameter tuning and model evaluation. The complexity and computational intensity of the methods, especially when applied to multiple datasets with various hyperparameters, significantly slowed down the experimentation process. This constraint limited the possibility of exploring a wider range of models, parameters, preprocessing methods, and datasets, which could potentially yield better results than those observed in this study.

Based on the identified limitations, future research can focus on improving the computational efficiency of models and parameter tuning processes. This can be achieved by redesigning the code part of these methods from a computer science perspective, leveraging advances in algorithm optimization, parallel computing, or hardware acceleration to significantly speed up training and inference. This work has revealed the possibility of creating a well-structured Python package with fast and efficient implementations of the GRNN and PNN models, including various modifications to improve usability and performance. Currently, existing implementations of these models are limited in functionality and do not support a wide range of modifications. Therefore, a logical next step and an important direction for future work is the development of a well-structured open source Python package that integrates various GRNN and PNN modifications.

In conclusion, all the objectives have been successfully achieved. The theoretical foundations have been thoroughly explored, the regularization methods have been adapted and implemented for GRNN and PNN models, and their performance has been rigorously evaluated on real data sets. Despite certain computational limitations, the study has provided valuable insights into the benefits and challenges of applying L1 and L2 regularization to nonparametric neural networks. The results provide a solid foundation for future research and practical applications in this area.

Bibliography

- Bishop, C. M. (1994). *Mixture density networks* (Working paper No. NCRG/94/004). Aston University.
- Bishop, C. M. (2006). *Pattern recognition and machine learning (information science and statistics)*. Springer-Verlag.
- Duda, R. O., Hart, P. E., & Stork, D. G. (2000). *Pattern classification (2nd edition)*. Wiley-Interscience.
- Efron, B., Hastie, T., Johnstone, I., & Tibshirani, R. (2004). Least angle regression. *The Annals of Statistics*, 32(2), 407–499. <https://doi.org/10.1214/009053604000000067>
- Fadel, S., Said, G., Abdallah, M., & Abu, H. (2016). Investigating the effect of different kernel functions on the performance of svm for recognizing arabic characters. *International Journal of Advanced Computer Science and Applications*, 7. <https://doi.org/10.14569/IJACSA.2016.070160>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <http://www.deeplearningbook.org>
- Han, J., Kamber, M., & Pei, J. (2011). *Data mining: Concepts and techniques* (3rd). Morgan Kaufmann Publishers Inc.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction*. Springer. <https://books.google.cz/books?id=eBSgoAEACAAJ>
- Hooda, N. (2018). Audit data [Dataset]. *UCI Machine Learning Repository*. <https://doi.org/10.24432/C5930Q>
- Chen, Y.-C. (2017). A tutorial on kernel density estimation and recent advances [Preprint]. <https://arxiv.org/abs/1704.03924>
- Kingma, D. P., & Ba, J. (2017). Adam: A method for stochastic optimization [Preprint]. <https://arxiv.org/abs/1412.6980>
- Little, M. (2007). Parkinsons [Dataset]. *UCI Machine Learning Repository*. <https://doi.org/10.24432/C59C74>
- Murphy, K. P. (2022). *Probabilistic machine learning: An introduction*. MIT Press. <http://probml.github.io/book1>
- Parzen, E. (1962). On Estimation of a Probability Density Function and Mode. *The Annals of Mathematical Statistics*, 33(3), 1065–1076. <https://doi.org/10.1214/aoms/1177704472>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Phillips, J. M., & Venkatasubramanian, S. (2011). A gentle introduction to the kernel distance [Preprint]. <https://arxiv.org/abs/1103.1625>

- Rosenblatt, M. (1956). Remarks on some nonparametric estimates of a density function. *The Annals of Mathematical Statistics*, 27(3), 832–837. <https://doi.org/10.1214/aoms/1177728190>
- Specht, D. F. (1990). Probabilistic neural networks. *Neural Networks*, 3(1), 109–118. [https://doi.org/10.1016/0893-6080\(90\)90049-Q](https://doi.org/10.1016/0893-6080(90)90049-Q)
- Specht, D. F. (1991). A general regression neural network. *IEEE Transactions on Neural Networks*, 2(6), 568–576. <https://doi.org/10.1109/72.97934>
- Specht, D. F. (1992). Enhancements to probabilistic neural networks. *[Proceedings 1992] IJCNN International Joint Conference on Neural Networks*, 1, 761–768 vol.1. <https://doi.org/10.1109/IJCNN.1992.287095>
- Specht, D. F. (1995). Probabilistic neural networks and general regression neural networks. In *Fuzzy logic and neural network handbook*. McGraw-Hill, Inc.
- Tsanas, A., & Xifara, A. (2012). Energy efficiency [Dataset]. *UCI Machine Learning Repository*. <https://doi.org/10.24432/C51307>
- Vinogradov, G. (2025). Regularization methods in probabilistic neural networks and general regression neural networks [GitHub repository]. <https://github.com/vinogradovvw/regularization-pnn-grnn>
- Wolberg, W. H., Street, W. N., & Mangasarian, O. L. (1993). Breast cancer wisconsin (diagnostic) [Dataset]. *UCI Machine Learning Repository*. <https://doi.org/10.24432/C5DW2B>
- Yeh, I.-C. (1998). Concrete compressive strength [Dataset]. *UCI Machine Learning Repository*. <https://doi.org/10.24432/C5PK67>
- Zeinali, Y., & Story, B. (2017). Competitive probabilistic neural network. *Integrated Computer - Aided Engineering*, 24, 1–14. <https://doi.org/10.3233/ICA-170540>

Appendices

A. Algorithms of optimization

Algorithm 1 Adam

Require: Initial parameters β' (random in this case), learning rate γ (0,001), Exponential decay rates β_1 (0,9), β_2 (0,999), Tolerance ϵ , small constant δ (e.g., 10^{-8})

Initialize first moment vector $m := 0$

Initialize second moment vector $v := 0$

Set timestep $t := 0$

while not converged **do**

$t := t + 1$

 Compute gradient: $g := \nabla f(\beta')$

 Update biased first moment estimate: $m := \beta_1 \cdot m + (1 - \beta_1) \cdot g$

 Update biased second moment estimate: $v := \beta_2 \cdot v + (1 - \beta_2) \cdot g^2$

 Compute bias-corrected first moment: $\hat{m} := \frac{m}{1 - \beta_1^t}$

 Compute bias-corrected second moment: $\hat{v} := \frac{v}{1 - \beta_2^t}$

 Update parameters: $\beta' := \beta' - \gamma \cdot \frac{\hat{m}}{\sqrt{\hat{v} + \delta}}$

if $\|\nabla f(\beta')\| < \epsilon$ **then**

break

end if

end while

return β'
